

PRO2 Lernskript - Finale Version

Klausurorientiertes Verständnis-Skript mit modernem Layout

Erstellt aus den bereitgestellten PRO2-Unterlagen

2026-07-09

Fokus	Umfang	Ausgelassen
Coding + tiefes Verständnis	Themen 1-9 und 11	Thema 10 / Backtracking

Finale Version und Relevanzfilter

Dieses Dokument ist die finale, modern gestaltete und klausurorientierte Version des Lernskripts. Es enthält:

1. eine Analyse der bereitgestellten Dateien,
2. eine vollständige Liste der prüfungsrelevanten Themen nach Relevanzfilter aus `PR0_Klausurinfo.md`,
3. ein detailliertes Inhaltsverzeichnis für das Gesamtskript,
4. die ausgearbeiteten Kapitel **1 bis 9 und 11**.

Thema 10 wird ausgelassen, weil Backtracking laut Klausurinfo nicht prüfungsrelevant ist.

Wichtigster Relevanzfilter: Das Skript orientiert sich zuerst an `PR0_Klausurinfo.md`. Die Folien liefern die fachliche Basis und die Beispielklausur `KLA_PR02_2025.pdf` liefert das Aufgabenformat.

Inhaltsverzeichnis

Dieses Inhaltsverzeichnis ist als schnelle Navigation gedacht und in ODT/PDF klickbar. Zusätzlich sind innerhalb des Skripts Querverweise gesetzt, zum Beispiel von `String.equals` zu `equals/hashCode`, von `Integer.parseInt` zu `Ausnahmebehandlung` und von `List<Integer>` zu `Adjazenzlisten`.

- Finale Version und Relevanzfilter
- 1. Quellenanalyse
- 2. Vollständige Liste der prüfungsrelevanten Themen
- 3. Detailliertes Inhaltsverzeichnis für das Gesamtskript
- Kapitel 1 - Einführung in Java
 - Strings: `==` vs. `equals`
 - Klassenmethoden
 - Parameterübergabe / call-by-value
 - Felder
- Kapitel 2 - Klassen und Objekte
 - Geheimnisprinzip
 - Konstruktoren
 - Beispielklasse `Rational`
- Kapitel 3 - Objektorientierte Programmierung
 - Vererbung und Faktorisierung
 - Dynamische Bindung
 - `Object`, `equals`, `hashCode`
 - Abstrakte Klassen und Interfaces
- Kapitel 4 - Datenorientierter Systementwurf und ATG
 - Terminalklassen
 - LL(1)-Bedingung
 - Attributierte Grammatiken
- Kapitel 5 - Ausnahmebehandlung
 - `try`, `catch`, `finally`
 - Checked vs. unchecked
- Kapitel 6 - Pakete, Generics und Java-Bibliothek
 - Pakete und Sichtbarkeit
 - Generics - Minimum
 - Bibliotheksmethoden
- Kapitel 7 - Hashing-basierte Suche
 - Kollisionen
 - Verkettung
 - Offene Adressierung
 - `equals` und `hashCode`
- Kapitel 8 - Heap-Datenstruktur
 - Feldrepräsentation
 - `swim`
 - `sink`
 - `removeMax`
- Kapitel 9 - Graphen
 - Adjazenzmatrix vs. Adjazenzliste
 - `AdjListDigraph`

- addEdge, removeEdge, reverse
- Kapitel 10 - ausgelassen: Backtracking
- Kapitel 11 - Objektorientierter Entwurf
 - UML-Basics
 - Abbott-Heuristik
 - Faktorisierung
 - Composite Pattern
- Gesamt-Prüfungsstrategie
- Finale Checkliste gegen typische Punkteverluste

| 1. Quellenanalyse

1.1 PRO_Klausurinfo.md - maßgeblicher Klausurfilter

Die Klausur ist nach den Notizen eine Mischung aus Theorie- und Praxisaufgaben, aber mit deutlichem Coding-Schwerpunkt. Kleine Implementierungen auf Papier sind realistisch. Eine Aufgabe soll ungefähr in 20 Minuten lösbar sein. Besonders wichtig sind daher nicht nur Definitionen, sondern das sichere Anwenden in kurzen Codeaufgaben.

Zentrale Hinweise aus der Klausurinfo:

- Praxis und Theorie etwa 50/50, Schwerpunkt Coding.
- Alle Themen können grundsätzlich Codingaufgaben enthalten.
- Falls Listen, Stacks oder ähnliche Behälter gebraucht werden, werden sie in der Aufgabe angegeben.
- `Integer.parseInt` darf verwendet werden und sollte bekannt sein.
- **ATG kommt sicher**, aber nicht als vollständige Parser-/Interpreter-Implementierung. Wichtig ist die Voraussetzung für rekursiven Abstieg, also LL(1)-Konformität.
- ATG-Aufgaben verwenden häufig `number` und `ident`; Terminalklassen müssen zumindest sprachlich oder als Regex beschrieben werden können.
- Einfache Klassen mit Konstruktoren und Methoden sind sehr wahrscheinlich.
- Backtracking kommt gar nicht, auch nicht theoretisch.
- Generics kommen nicht bzw. nur minimal.
- Heapsort kommt nicht; die Heap-Datenstruktur selbst ist relevant.
- Bei Entwurfsmustern ist **Composite** relevant, andere Patterns nicht.

1.2 Folien 01-11 - fachliche Basis

Die Folien geben die fachlichen Details. Für das Skript werden sie aber nicht ungefiltert übernommen. Stattdessen werden die Inhalte auf Klausurtauglichkeit reduziert:

- **01 Einführung in Java:** Java-Plattform, statische Typisierung, primitive und Referenztypen, Strings, Eingabe/Konvertierung, Anweisungen, Klassenmethoden, call-by-value, Felder.
- **02 Klassen und Objekte:** Objekt-/Klassenbegriff, Methoden, Konstruktoren, `this`, constructor chaining, Sichtbarkeit, Geheimnisprinzip, einfache Klassen wie Stack/Color.
- **03 OOP:** Vererbung, Überschreiben, `super`, Konstruktorverkettung, `Object`, Polymorphismus, dynamische Bindung, abstrakte Klassen, Interfaces, `instanceof/Cast`, Vererbungsarten.
- **04 ATG:** Datenorientierter Entwurf, Grammatiknotation, LL(1), attributierte Grammatiken, Terminalklassen, semantische Aktionen, Entwurfsweg vom Problem zur ATG.
- **05 Ausnahmebehandlung:** Exception-Objekte, `try/catch/finally`, checked/unchecked, `throws`, eigene Exceptions.
- **06 Java-Bibliothek:** Pakete/Importe und ausgewählte Bibliotheksmethoden, Generics nur minimal.
- **07 Hashing:** Hashfunktion, Kollisionen, Verkettung/offene Adressierung, Lastfaktor, `equals/hashCode`.
- **08 Heap:** Priority Queue, binärer Heap, Feld-Baum-Abbildung, Heap-Ordnung, `swim/sink`; Heapsort nicht.
- **09 Graphen:** Begriffe, Adjazenzmatrix/-liste, gerichtete Graphen in Java, einfache Methoden auf Adjazenzlisten.
- **11 OO Entwurf:** UML-Klassendiagramme, erster Klassenentwurf aus Text, Faktorisierung, Composite Pattern.

1.3 Beispielklausur KLA_PRO2_2025.pdf - typisches Aufgabenformat

Die Beispielklausur bestätigt den Stil der Klausur: kurze, punktgenaue Aufgaben, die Verständnis und kleine Implementierungen kombinieren. Enthalten sind:

1. **Felder:** Array-/Referenzstruktur zeichnen und eine `join`-Klassenmethode implementieren.
2. **Methoden und Klassenmethoden:** Unterschiede erklären und Parameterübergabe an primitiven Typen, `String` und Arrays skizzieren.
3. **Heap-Datenstruktur:** Feld-Baum-Abbildung und Wiederherstellung der Heap-Ordnung.
4. **Attributierte Grammatiken:** ATG für eine Hotelbewertungs-Sprache entwickeln.
5. **Klasse Polygon:** einfache Klasse implementieren und `equals(Object o)` überschreiben.
6. **Vererbung:** Begriff, Zweck und Java-Vererbungsarten erklären.
7. **Ausnahmebehandlung:** Ausgabe bei verschiedenen Exception-Fällen bestimmen.

Daraus folgt für die Vorbereitung: **Zeichnen, kleine Methoden, kleine Klassen, ATG und Code-Tracing** müssen aktiv geübt werden. Reines Auswendiglernen reicht nicht.

| 2. Vollständige Liste der prüfungsrelevanten Themen

Thema 1 - Einführung in Java

Relevant: Java vs. Pascal/klassisch kompilierte Sprache; Bytecode und JVM; statische Typisierung; robuste Programme; primitive und Referenztypen; Strings; `==` vs. `equals`; Defaultwerte; lokale Variablen; Eingabe und `Integer.parseInt`; Anweisungen; Klassenmethoden; call-by-value; Überladen; Felder.

Typische Aufgaben: Ausgaben vorhersagen, Speicher-/Referenzdiagramme zeichnen, eine Klassenmethode wie `join`, `reverse`, `max`, `count` oder `parse` schreiben, Parameterübergabe skizzieren.

Thema 2 - Klassen und Objekte

Relevant: Objektbegriff, Klassenbegriff, Datenkomponenten/Methoden, Konstruktoren, `this`, constructor chaining mit `this(...)`, Geheimnisprinzip, `private` vs. `public`, einfache Klassenimplementierung.

Typische Aufgaben: Klasse für Punkt, Farbe, Bruchzahl, Dreieck, Stack oder Polygon entwerfen; Konstruktoren und Methoden implementieren; invariantenbewusst programmieren.

Thema 3 - Objektorientierte Programmierung

Relevant: Vererbung, Faktorisierung, Überschreiben vs. Überladen, `super`, Konstruktoren bei Vererbung, `Object`, `toString`, `equals`, `hashCode`, Polymorphismus, statischer/dynamischer Typ, dynamische Bindung, abstrakte Klassen, Interfaces, `instanceof` und Casts, Nachbilden von Mehrfachvererbung.

Typische Aufgaben: Code-Tracing mit dynamischer Bindung, Vererbungsfragen erklären, `equals/hashCode` implementieren, UML lesen/zeichnen, abstrakte Klasse vs. Interface begründen.

Thema 4 - Datenorientierter Systementwurf / ATG

Relevant: Grammatiknotation, Terminalsymbole/Terminalklassen, Nichtterminale, EBNF-Operatoren, LL(1)-Bedingung, Umformen nicht-LL(1)-konformer Regeln, attributierte Grammatiken, Eingangs-/Ausgangsattribute, semantische Aktionen.

Nicht relevant im Detail: vollständige Parser-/Interpreter-Implementierung.

Typische Aufgaben: Eine Grammatik/ATG für ein kleines Textformat entwickeln; `number` und `ident` als Terminalklassen beschreiben; LL(1)-Probleme erkennen und Regeln umschreiben.

Thema 5 - Ausnahmebehandlung

Relevant: Exception-Objekte, Exception-Hierarchie, `try/catch/finally`, checked vs. unchecked Exceptions, `throws`, Catch-Reihenfolge, `finally`-Ablauf, eigene Exceptions, Eingabevalidierung.

Typische Aufgaben: Ausgabe eines Codefragments bestimmen; entscheiden, ob `throws` nötig ist; robuste Methode mit Validierung schreiben.

Thema 6 - Pakete, Generics und Java-Bibliothek

Relevant: Pakete, Sichtbarkeit über Paketgrenzen, `public`, `import`, wichtige Bibliotheksklassen/-methoden, insbesondere Konvertierungsmethoden wie `Integer.parseInt`.

Reduziert: Generische Typen kommen laut Klausurinfo nicht bzw. nur minimal.

Thema 7 - Hashing-basierte Suche

Relevant: Hashing-Idee, Hashcodes, Kollisionen, warum Kollisionen unvermeidbar sind, Kollisionsbehandlung durch Verkettung und offene Adressierung, Lastfaktor/Füllstand, gute Hashfunktion, Zusammenhang mit equals/hashCode.

Thema 8 - Heap-Datenstruktur

Relevant: Priority Queue, kompletter Binärbaum, Feld-Baum-Abbildung mit freier Position 0, Heap-Eigenschaft, swim, sink, insert, removeMax, Fehler in Heap erkennen und beheben.

Nicht relevant: Heapsort.

Thema 9 - Graphen

Relevant: Graphbegriffe, gerichtete Graphen, Grad/Eingangsgrad/Ausgangsgrad, Weg/Zyklus/Zusammenhang, Adjazenzmatrix vs. Adjazenzliste, einfache Java-Methoden auf Adjazenzlisten.

Thema 10 - Backtracking

Nicht prüfungsrelevant. Wird vollständig ausgelassen.

Thema 11 - Objektorientierter Entwurf

Relevant: Textanalyse für ersten Klassenentwurf, UML-Klassendiagramme lesen/zeichnen, Klassen/Methoden/Datenkomponenten identifizieren, Beziehungen, Faktorisierung, Composite Pattern.

Reduziert: andere Design Patterns werden nicht vertieft.

3. Detailliertes Inhaltsverzeichnis für das Gesamtskript

Kapitel 1 - Einführung in Java

Relevante Dateien/Folien: 01 Einführung in Java (V 11.1).pdf, PR0_Klausurinfo.md, KLA_PR02_2025.pdf Aufgaben 1 und 2.

Klausurwichtige Konzepte: Java = Sprache + JVM + Bibliothek; Bytecode; statische Typisierung; primitive vs. Referenztypen; Strings; Defaultwerte; lokale Variablen; Klassenmethoden; call-by-value; Felder.

Typische Fragen:

- Warum ist Java robuster als eine einfache statisch typisierte Sprache ohne Exception-Konzept?
- Was ist der Unterschied zwischen `s == p` und `s.equals(p)`?
- Was passiert bei `int[] b = a; b[5] = 2;`?
- Warum kann eine Methode ein Array-Inhalt ändern, aber nicht die Variable des Aufrufers auf ein anderes Array „umhängen“?

Kapitel 2 - Klassen und Objekte

Relevante Dateien/Folien: 02 Klassen und Objekte.pdf, PR0_Klausurinfo.md, KLA_PR02_2025.pdf Aufgabe 5.

Klausurwichtige Konzepte: Objekt als Daten + Operationen; Klasse als Bauplan; Konstruktoren; `this`; constructor chaining; `private` ist klassenbezogen; einfache Klassen implementieren.

Typische Fragen:

- Implementieren Sie eine Klasse `Rational` mit Konstruktor und Multiplikation.
- Welche Unterschiede haben Objektmethode und Klassenmethode?
- Warum schützt `private` nicht vor Zugriffen durch andere Objekte derselben Klasse?

Kapitel 3 - Objektorientierte Programmierung

Relevante Dateien/Folien: 03 OOP (V 11.1).pdf, PR0_Klausurinfo.md, KLA_PR02_2025.pdf Aufgabe 6.

Klausurwichtige Konzepte: Vererbung, Faktorisierung, Überschreiben, Überladen, `super`, `Object`, `equals`/`hashCode`, Polymorphismus, dynamische Bindung, abstrakte Klassen, Interfaces.

Typische Fragen:

- Welche Methode wird bei dynamischer Bindung aufgerufen?
- Warum benötigt `equals(Object o)` meist `instanceof` und `Cast`?
- Wie kann Java Mehrfachvererbung nachbilden?

Kapitel 4 - Attributierte Grammatiken

Relevante Dateien/Folien: 04 ATG.pdf, PR0_Klausurinfo.md, KLA_PR02_2025.pdf Aufgabe 4.

Klausurwichtige Konzepte: Grammatiknotation, Terminalklassen, LL(1), attributierte Grammatik, semantische Aktionen, einfache Transformationen von Eingabedatenströmen.

Typische Fragen:

- Ist die Regel `B = C | C "." C`, LL(1)-tauglich? Falls nein: umformen.
- Beschreiben Sie `ident` und `number` sprachlich oder per Regex.
- Entwickeln Sie eine ATG, die eine Zähl- oder Summenaufgabe löst.

Kapitel 5 - Ausnahmebehandlung

Relevante Dateien/Folien: 05 Ausnahmebehandlung.pdf, PR0_Klausurinfo.md, KLA_PR02_2025.pdf Aufgabe 7.

Klausurwichtige Konzepte: Exception-Hierarchie, checked/unchecked, try/catch/finally, throw, throws, Catch-Reihenfolge.

Typische Fragen:

- Welche Ausgabe erzeugt ein Codefragment mit try, mehreren catch und finally?
- Muss eine Methode throws IOException deklarieren?
- Welche Validierung gehört in public Methoden?

Kapitel 6 - Pakete, Generics und Bibliothek

Relevante Dateien/Folien: 06 Java-Bibliothek.pdf, PRO_Klausurinfo.md.

Klausurwichtige Konzepte: Paketgrenzen, public, Import, Java-Bibliothek, Integer.parseInt; Generics nur minimal.

Typische Fragen: Sichtbarkeit über Paketgrenzen, richtige Verwendung einfacher Bibliotheksmethoden.

Kapitel 7 - Hashing

Relevante Dateien/Folien: 07 Hashing.pdf, PRO_Klausurinfo.md.

Klausurwichtige Konzepte: Hashfunktion, Kollision, Verkettung, offene Adressierung, Lastfaktor, gute Hashfunktion.

Typische Fragen: Warum sind Kollisionen unvermeidbar? Wann ist Suche annähernd $O(1)$? Wie implementiert man hashCode passend zu equals?

Kapitel 8 - Heap

Relevante Dateien/Folien: 08 Heap.pdf, PRO_Klausurinfo.md, KLA_PRO2_2025.pdf Aufgabe 3.

Klausurwichtige Konzepte: Heap als kompletter Binärbaum im Feld, Index 0 frei, Heap-Ordnung, swim, sink.

Typische Fragen: Feld zu Baum zeichnen; Baum zu Feld zeichnen; verletzte Heap-Ordnung finden und beheben.

Kapitel 9 - Graphen

Relevante Dateien/Folien: 09 Graphen.pdf, PRO_Klausurinfo.md.

Klausurwichtige Konzepte: Graphbegriffe, gerichtete Graphen, Adjazenzlistenrepräsentation, einfache Methoden.

Typische Fragen: Methode addEdge, removeEdge, outdegree, indegree, reverse skizzieren.

Kapitel 11 - Objektorientierter Entwurf

Relevante Dateien/Folien: 11 00 Entwurf.pdf, PRO_Klausurinfo.md.

Klausurwichtige Konzepte: UML lesen/zeichnen, Klassen aus Text identifizieren, Faktorisierung, Composite Pattern.

Typische Fragen: Aus einer Aufgabenbeschreibung einen ersten Klassenentwurf ableiten; Klassenhierarchie und Beziehungen als UML skizzieren.

Kapitel 1 - Einführung in Java

[Zurück zur Kapitelübersicht](#)

1.1 Klausurfokus

Dieses Kapitel ist vor allem relevant für kurze Verständnisfragen und kleine Codingaufgaben. Besonders klausurnah sind:

- Unterschied zwischen Java und Pascal bzw. klassisch kompilierten Sprachen,
- statische Typisierung und Fehlererkennung durch den Compiler,
- Robustheit durch Laufzeitprüfungen und Ausnahmebehandlung,
- primitive Typen vs. Referenztypen,
- Zeichenkettenvergleich,
- Defaultwerte und lokale Variablen,
- Klassenmethoden und Parameterübergabe,
- Felder, insbesondere Referenzen und Alias-Effekte.

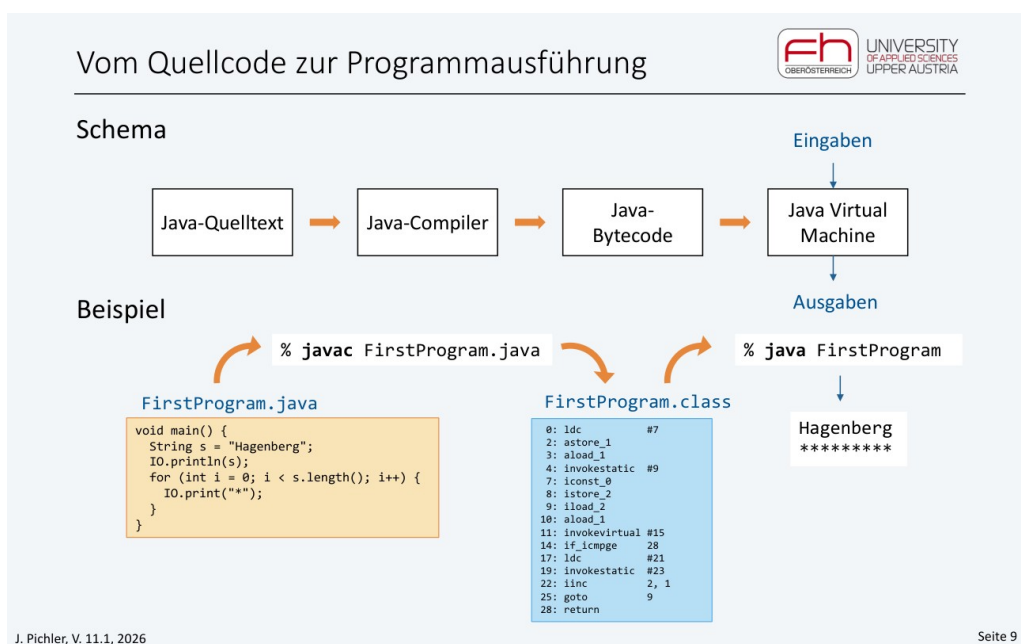
Die Beispielklausur enthält genau diese Richtung: Array-Strukturen zeichnen, eine `join`-Methode schreiben und Parameterübergabe erklären.

1.2 Java als Ausführungsmodell: Quelltext, Bytecode, JVM

Java besteht nicht nur aus der Programmiersprache. Für die Klausur ist wichtig, Java als Kombination aus **Sprache**, **virtueller Maschine** und **Bibliothek** zu verstehen.

```
Java-Quelltext (.java)
  ↓ Compiler javac
Java-Bytecode (.class)
  ↓ Java Virtual Machine
Maschinenausführung auf konkretem Rechner
```

Der Compiler übersetzt Java-Quelltext nicht direkt in nativen Maschinencode für genau einen Prozessor, sondern in **Bytecode**. Dieser Bytecode wird von der **Java Virtual Machine (JVM)** ausgeführt. Dadurch ist Java architekturneutraler und portabler: derselbe Bytecode kann auf verschiedenen Plattformen laufen, solange dort eine passende JVM vorhanden ist.



Vom Quellcode zur Programmausführung. Quelle: 01 Einführung in Java, Folie 9.

Prüfungsverständnis:

- Pascal wurde in den Grundlagen oft als Beispiel für eine klassisch kompilierte Sprache betrachtet: Quellcode wird in maschinennahen Code übersetzt.
- Java trennt stärker zwischen Kompilierung und Ausführung: Der Compiler erzeugt Bytecode; die JVM führt Bytecode aus.
- Beide Sprachen sind statisch typisiert: Der Typ eines Ausdrucks/Objekts ist zur Übersetzungszeit kontrollierbar.

1.3 Statische Typisierung und Robustheit

Statisch typisiert bedeutet: Jede Variable, jeder Ausdruck und jede Methode hat einen Typ, der vom Compiler geprüft wird. Dadurch erkennt der Compiler viele Fehler vor dem Start des Programms.

Beispiele:

```
int x = 3;
String s = "Hallo";
x = s;           // Compilerfehler: String passt nicht in int
```

Java ist zusätzlich robust, weil viele gefährliche Situationen zur Laufzeit definiert behandelt werden:

- Zugriff auf Array-Index außerhalb der Grenzen löst eine Exception aus.
- Division durch 0 bei Integer-Werten löst eine `ArithmeticException` aus.
- `null`-Zugriff löst eine `NullPointerException` aus.
- Es gibt automatische Speicherbereinigung statt manuellem `delete`.
- Ausnahmebehandlung ermöglicht kontrolliertes Reagieren auf Fehler. Mehr dazu in [Kapitel 5](#).

Klausurformulierung: Wenn gefragt wird, ob Java oder Pascal robuster ist, ist die erwartete Richtung: Java, insbesondere wegen Ausnahmebehandlung, Laufzeitprüfungen und automatischer Speicherbereinigung. Statische Typisierung allein haben beide.

1.4 Primitive Typen vs. Referenztypen

Java unterscheidet zwischen primitiven Typen und Referenztypen.

Primitive Typen speichern den Wert direkt:

```
int a = 5;
boolean ok = true;
char c = 'x';
```

Referenztypen speichern eine Referenz auf ein Objekt oder Array am Heap:

```
String s = "Hallo";
int[] a = new int[7];
Rational r = new Rational(1, 2);
```

Das ist entscheidend für viele Klausuraufgaben. Eine Variable vom Typ `int[]` enthält nicht das ganze Array, sondern eine Referenz auf das Array.

```
int[] a = new int[7];
int[] b = a;
a[3] = 1;
b[5] = 2;
```

Nach diesen Anweisungen zeigen `a` und `b` auf dasselbe Array. Deshalb stehen im einen Array an Index 3 der Wert 1 und an Index 5 der Wert 2. Es gibt **kein zweites Array**.

1.5 Defaultwerte und lokale Variablen

Bei Arrays und Objekt-Datenkomponenten gibt es Defaultwerte. Lokale Variablen sind dagegen nicht automatisch sinnvoll initialisiert.

Ort / Art	Initialisierung
<code>int[] a = new int[5]</code>	alle Elemente 0
<code>boolean[] b = new boolean[5]</code>	alle Elemente false
<code>String[] s = new String[5]</code>	alle Elemente null
Datenkomponente <code>private int x;</code>	0
lokale Variable <code>int x;</code>	nicht verwendbar, bis explizit initialisiert

Erzeugen eines Feldes



- Elemente werden mit Default-Wert des Elementtyps initialisiert (0, 0.0, false, null)

```
String[] s = new String[5];
```

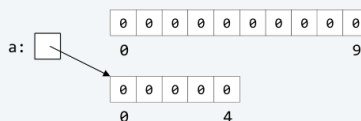


- mehrmalige Zuweisung möglich

```
int[] a;
```

```
a = new int[10];
```

```
a = new int[5];
```



Erzeugen eines Feldes durch Angabe von Initialwerten

```
int[] fib = new int[] {0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55};
```

```
int[] fib = {0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55};
```

J. Pichler, V. 11.1, 2026

Seite 67

Felder werden beim Erzeugen mit Defaultwerten initialisiert. Quelle: 01 Einführung in Java, Folie 67.

Typischer Fehler:

```
public static int f(boolean b) {
    int x;
    if (b) {
        x = 1;
    }
    return x;    // Compilerfehler: x ist eventuell nicht initialisiert
}
```

Der Compiler verlangt, dass lokale Variablen auf jedem möglichen Ausführungspfad initialisiert sind.

1.6 Zeichenketten: Referenzvergleich vs. Inhaltsvergleich

String ist ein Referenztyp. Deshalb muss man zwei Ebenen unterscheiden:

```
String s = "Hallo";
String p = "Hallo";
```

- `s == p` vergleicht Referenzen: Zeigen beide Variablen auf dasselbe Objekt?
- `s.equals(p)` vergleicht Inhalte: Haben beide Strings dieselbe Zeichenfolge?

Robustheitsfalle:

```
if (s.equals(p)) { ... }
```

Dieser Ausdruck ist korrekt, falls `s != null`. Falls `s == null`, führt der Methodenaufzuruf zu einer `NullPointerException`.

Robuster ist oft:

```
if (p != null && p.equals(s)) { ... }
```

oder bei Vergleich mit Konstante:

```
if ("Hallo".equals(s)) { ... }
```

Klausurpunkt: `s == p` ist nicht „falsch“ im Sinn eines Laufzeitfehlers. Es ist robust gegen `null`, prüft aber nur Referenzgleichheit, nicht Inhaltsgleichheit.

Siehe dazu auch `equals(Object o)` in Kapitel 3, wo die Methode für eigene Klassen überschrieben wird.

1.7 Eingabe, Konvertierung und `Integer.parseInt`

Eingabe kommt häufig als Zeichenkette. Soll daraus eine Zahl werden, wird konvertiert:

```
String line = IO.readLine("Zahl = ");
int value = Integer.parseInt(line);
```

`Integer.parseInt` ist klausurrelevant. Es darf verwendet werden und sollte bekannt sein. Die Methode kann eine `NumberFormatException` auslösen, wenn der Text keine gültige ganze Zahl enthält.

```
int a = Integer.parseInt("42"); // 42
int b = Integer.parseInt("abc"); // NumberFormatException
```

Die Exception-Seite dazu steht in Kapitel 5.

1.8 Anweisungen und Kontrollfluss

Die grundlegenden Anweisungen aus PRO1 bleiben relevant, aber meist eingebettet in Java-Code:

```
if (x > 0) {
    IO.println("positiv");
} else {
    IO.println("nicht positiv");
}

for (int i = 0; i < a.length; i++) {
    IO.println(a[i]);
}

while (i < a.length && a[i] != value) {
    i++;
}
```

Für eine sehr gute Note ist wichtig, Schleifen nicht nur syntaktisch zu können, sondern **Invarianten** und Grenzen sauber zu behandeln:

- Beginnt der Index bei 0?
- Ist die Abbruchbedingung `< a.length` und nicht `<= a.length`?
- Wird `null` korrekt übersprungen?
- Wird ein Trennzeichen nur zwischen Elementen geschrieben und nicht am Ende?

1.9 Klassenmethoden und Aufbau einer Methode

Eine Klassenmethode ist mit `static` deklariert und wird auf der Klasse aufgerufen:

```
public static int max(int a, int b) {
    return (a > b) ? a : b;
}

int m = Math.max(3, 7);
```

Eine Methode besteht aus:

- Rückgabetyp,
- Name,
- Parameterliste,
- Rumpf,
- optionalem `return`.

```
public static int gcd(int p, int q) {
    int r = p % q;
    while (r != 0) {
        p = q;
        q = r;
        r = p % q;
    }
    return q;
}
```

Überladen bedeutet: mehrere Methoden haben denselben Namen, aber verschiedene Parameterlisten.

```
public static int max(int a, int b) { ... }
public static int max(int a, int b, int c) { ... }
public static double max(double a, double b) { ... }
```

Der Compiler entscheidet anhand der Argumenttypen, welche Methode gemeint ist.

1.10 Parameterübergabe: Java verwendet call-by-value

Java übergibt Parameter immer **by value**. Das gilt auch bei Referenztypen. Entscheidend ist: Bei Referenztypen wird der Referenzwert kopiert.

Primitive Werte

```
public static void swap(int a, int b) {
    int t = a;
    a = b;
    b = t;
}

int x = 1;
int y = 2;
swap(x, y);
// x == 1, y == 2
```

`swap` vertauscht nur die lokalen Kopien `a` und `b`.

Referenzen auf unveränderliche Objekte wie `String`

```
public static void swap(String a, String b) {
    String t = a;
    a = b;
    b = t;
}
```

Auch hier werden nur die lokalen Referenzkopien vertauscht. Die Variablen des Aufrufers ändern sich nicht.

Arrays

```
public static void swapFirst(int[] a, int[] b) {
    int t = a[0];
    a[0] = b[0];
    b[0] = t;
}
```

Hier werden zwar die Referenzen selbst nur kopiert, aber beide Kopien zeigen auf dieselben Array-Objekte wie beim Aufrufer. Deshalb ändern sich die Array-Inhalte.

Merksatz: Eine Methode kann nicht die Variable des Aufrufers neu binden, aber sie kann über eine kopierte Referenz das referenzierte Objekt verändern.

1.11 Felder: Erzeugen, Zugriff, Referenzen

Felder sind Objekte am Heap. Die Feldvariable enthält eine Referenz.

```
int[] a = new int[5];
a[0] = 10;
```

```
a[4] = 20;
IO.println(a.length);    // 5
```

Zugriffe werden zur Laufzeit geprüft:

```
a[-1] = 3;    // ArrayIndexOutOfBoundsException
a[5] = 3;    // ArrayIndexOutOfBoundsException
```

Referenzdiagramme verstehen

```
String s = "Hallo";
String[] a = new String[5];
a[0] = s;
a[2] = s;
a[4] = a[0] + "Hallo";
```

Nachher gilt:

- `a[0]` und `a[2]` referenzieren dasselbe String-Objekt wie `s`.
- `a[1]` und `a[3]` sind `null`.
- `a[4]` referenziert ein neues String-Objekt mit Inhalt "HalloHallo".
- `a[0] == a[2]` ist sehr wahrscheinlich `true`, weil beide dieselbe Referenz aus `s` erhalten haben.
- `a[4].equals("HalloHallo")` ist `true`.

1.12 Klausurnahe Methode: `join`

Eine typische Aufgabe ist die Implementierung einer Klassenmethode, die ein Array verarbeitet. Beispiel aus der Klausurform:

```
String[] words = {"Eins", "Zwei", null, "Drei"};
String result = join(words, ',');
IO.println(result);    // Eins,Zwei,Drei
```

Eine saubere Lösung:

```
public static String join(String[] text, char delimiter) {
    String result = "";
    boolean first = true;

    for (int i = 0; i < text.length; i++) {
        if (text[i] != null) {
            if (!first) {
                result = result + delimiter;
            }
            result = result + text[i];
            first = false;
        }
    }

    return result;
}
```

Warum ist `first` wichtig? Das Trennzeichen soll **zwischen** den gültigen Elementen stehen, nicht am Anfang und nicht am Ende. `null`-Elemente sollen übersprungen werden.

Alternative mit `StringBuilder`:

```
public static String join(String[] text, char delimiter) {
    StringBuilder result = new StringBuilder();

    for (int i = 0; i < text.length; i++) {
        if (text[i] != null) {
            if (result.length() > 0) {
                result.append(delimiter);
            }
            result.append(text[i]);
        }
    }

    return result.toString();
}
```



```
    return result.toString();
}
```

Die erste Variante ist für Papierklausuren oft einfacher. Die zweite ist effizienter, weil nicht bei jeder Verkettung ein neuer String entsteht.

1.13 Mini-Check Kapitel 1

1. Zeichne die Speicherstruktur nach:

```
int[] a = new int[7];
int[] b = a;
a[3] = 1;
b[5] = 2;
```

2. Erkläre den Unterschied:

```
s == p
s.equals(p)
```

3. Was gibt folgender Code aus?

```
public static void m(int[] a) {
    a[0] = 99;
    a = new int[3];
    a[0] = 42;
}

int[] x = new int[2];
m(x);
IO.println(x[0]);
```

Lösungsidee: Ausgabe 99, weil nur der lokale Parameter `a` auf ein neues Array gesetzt wird. Das ursprüngliche Array wurde vorher aber verändert.

Kapitel 2 - Klassen und Objekte

[Zurück zur Kapitelübersicht](#)

2.1 Klausurfokus

Hier geht es um das Implementieren kleiner Klassen. Aus der Klausurinfo sind besonders wichtig:

- einfache Klassen mit Konstruktor und Methoden,
- Konstruktor ruft anderen Konstruktor mit `this(...)` auf,
- `private` und `public` sauber unterscheiden,
- `private` bezieht sich auf die Klasse, nicht nur auf ein Objekt,
- objektorientierte statt unnötig statische Lösung wählen.

2.2 Objekt und Klasse

Ein Objekt ist eine Laufzeitkomponente mit:

- Datenkomponenten, also Zustand,
- Methoden, also Operationen auf diesem Zustand.

Eine Klasse ist der Bauplan für solche Objekte.

```
public class Point {
    private int x;
    private int y;

    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public void moveBy(int dx, int dy) {
        x = x + dx;
        y = y + dy;
    }
}
```

Die Objektvariable enthält wieder nur eine Referenz:

```
Point p = new Point(3, 4);
```

`new Point(3, 4)` erzeugt ein Objekt. `p` referenziert dieses Objekt.

2.3 Datenkapselung und Geheimnisprinzip

Das Geheimnisprinzip besagt: Der direkte Zugriff auf den inneren Zustand eines Objekts soll verhindert werden. Von außen soll man nur über die öffentliche Schnittstelle arbeiten.

Geheimnisprinzip (*information hiding*)



- direkter Zugriff auf die Komponenten eines Objekts soll verhindert werden
- Zugriffsrechte werden z.B. durch `private` und `public` definiert

```
public class Rectangle {
    private int x, y, width, height;

    public Rectangle(int x, int y, int width, int height) { ... }
    public void moveBy(int dx, int dy) { ... }
    public boolean contains(int px, int py) { ... }
    public void draw() { ... }
}
```

`private` ... nur innerhalb der Klasse sichtbar

`public` ... global sichtbar

- Verwendung

```
Rectangle r1 = new Rectangle(10, 20, 100, 200);
r1.width = 42;
System.out.print(r1.x);
```

wegen `private` nicht möglich!

J. Pichler, V. 11.1, 2026

Seite 16

Geheimnisprinzip und Zugriff über öffentliche Methoden. Quelle: 02 Klassen und Objekte, Folie 16.

```
public class Account {
    private int balance;

    public void deposit(int amount) {
        if (amount > 0) {
            balance = balance + amount;
        }
    }

    public int getBalance() {
        return balance;
    }
}
```

Warum nicht `public int balance`? Weil dann jeder fremde Code beliebige, auch ungültige Zustände erzeugen könnte:

```
account.balance = -1_000_000; // bei public möglich, fachlich unsinnig
```

2.4 `private` ist klassenbezogen

Ein häufiger Denkfehler ist: `private` bedeute „nur dieses Objekt darf zugreifen“. In Java bedeutet es aber: **nur Code innerhalb derselben Klasse darf zugreifen**. Dieser Code darf auch auf `private` Komponenten anderer Objekte derselben Klasse zugreifen.

```
public class Point {
    private int x;
    private int y;

    public boolean samePosition(Point other) {
        return this.x == other.x && this.y == other.y;
    }
}
```

`other.x` ist erlaubt, weil der Code in der Klasse `Point` steht.

2.5 Konstruktoren und `this`

Ein Konstruktor initialisiert ein Objekt beim Erzeugen.

```
public class Rectangle {
    private int x;
    private int y;
    private int width;
```

```

private int height;

public Rectangle(int x, int y, int width, int height) {
    this.x = x;
    this.y = y;
    this.width = width;
    this.height = height;
}
}

```

`this` bezeichnet das aktuelle Empfängerobjekt. Es ist besonders wichtig, wenn Parameter gleich heißen wie Datenkomponenten.

Constructor chaining

Ein Konstruktor kann einen anderen Konstruktor derselben Klasse mit `this(...)` aufrufen. Der Aufruf muss die erste Anweisung im Konstruktor sein.

```

public class Rectangle {
    private int x;
    private int y;
    private int width;
    private int height;

    public Rectangle(int width, int height) {
        this(0, 0, width, height);
    }

    public Rectangle(int x, int y, int width, int height) {
        this.x = x;
        this.y = y;
        this.width = width;
        this.height = height;
    }
}

```

Klausurpunkt: Konstruktoren dürfen überladen werden. `this(...)` vermeidet Codeverdopplung bei Initialisierung.

2.6 Objektmethoden vs. Klassenmethoden

Eine **Objektmethode** arbeitet auf einem konkreten Objekt und hat implizit `this`.

```
r.moveBy(3, 5);
```

Eine **Klassenmethode** ist `static` und gehört zur Klasse, nicht zu einem Objekt.

```
int x = Math.max(3, 5);
```

Für fachliche Datentypen ist eine Objektmethode oft die bessere OO-Lösung:

```

public Rational multiply(Rational other) {
    return new Rational(this.nom * other.nom, this.denom * other.denom);
}

```

Eine rein statische Lösung ist manchmal möglich, aber weniger objektorientiert:

```

public static Rational multiply(Rational r1, Rational r2) {
    return new Rational(r1.nom * r2.nom, r1.denom * r2.denom);
}

```

2.7 Beispielklasse Rational

Eine klausurnahe Klasse für rationale Zahlen:

```

public class Rational {
    private int nom;
    private int denom;

    public Rational(int nom, int denom) {
        if (denom == 0) {

```

```

        throw new IllegalArgumentException("denom must not be 0");
    }
    this.nom = nom;
    this.denom = denom;
    normalizeSign();
}

public Rational multiply(Rational r) {
    return new Rational(this.nom * r.nom, this.denom * r.denom);
}

public void multiplyInPlace(Rational r) {
    this.nom = this.nom * r.nom;
    this.denom = this.denom * r.denom;
    normalizeSign();
}

private void normalizeSign() {
    if (denom < 0) {
        nom = -nom;
        denom = -denom;
    }
}
}

```

Verständnis:

- `multiply` mit Rückgabe ist funktionaler: Das Original bleibt unverändert.
- `multiplyInPlace` verändert das aktuelle Objekt.
- Eine statische `multiply(r1, r2)`-Methode ist möglich, aber weniger objektorientiert, weil die Operation nicht als Nachricht an ein Rational-Objekt modelliert ist.

2.8 Invarianten und Validierung

Eine Invariante ist eine Bedingung, die für jedes gültige Objekt immer gelten soll.

Bei Rational:

```
denom != 0
```

Bei Color:

```
0 <= red, green, blue <= 255
```

Bei Polygon:

```
0 <= size <= 100
```

Validierung ist besonders bei public Methoden wichtig, weil fremder Code sie mit ungültigen Werten aufrufen kann. Bei privaten Methoden kann man oft voraussetzen, dass sie nur kontrolliert aus der Klasse selbst aufgerufen werden. Wenn die Aufgabe ausdrücklich Validierung verlangt, muss sie eingebaut werden.

2.9 Mini-Check Kapitel 2

1. Warum ist `private` trotzdem mit `other.x` vereinbar, wenn `other` ein Objekt derselben Klasse ist?
2. Implementiere einen Konstruktor `Point()` der `Point(0, 0)` aufruft.
3. Was ist objektorientierter?

```

r1.multiply(r2)
Rational.multiply(r1, r2)

```

Lösungsidee: `r1.multiply(r2)`, weil die Operation als Nachricht an ein Objekt modelliert ist.

Kapitel 3 - Objektorientierte Programmierung

[Zurück zur Kapitelübersicht](#)

3.1 Klausurfokus

Dieses Kapitel ist klausurstark. Rechnen muss man vor allem mit:

- Vererbung erklären,
- Überschreiben und Überladen unterscheiden,
- dynamische Bindung in Codebeispielen auswerten,
- Object-Methoden sinnvoll überschreiben,
- equals und hashCode korrekt zusammendenken,
- abstrakte Klassen und Interfaces unterscheiden,
- Mehrfachvererbung durch Interfaces nachbilden,
- UML-Grundnotation verstehen.

3.2 Vererbung und Faktorisierung

Vererbung erlaubt, Gemeinsamkeiten in eine Basisklasse zu ziehen. Das nennt man **Faktorisierung**.

```
public class Person {
    private String name;

    public Person(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}

public class Student extends Person {
    private String id;

    public Student(String name, String id) {
        super(name);
        this.id = id;
    }
}
```

Student erbt die Eigenschaften und Methoden von Person und ergänzt die Matrikelnummer.

Wozu?

- Codeverdopplung vermeiden,
- gemeinsame Schnittstelle definieren,
- getesteten Code wiederverwenden,
- Systeme leichter erweitern.

3.3 Überschreiben vs. Überladen

Überladen: gleicher Methodename, andere Parameterliste. Entscheidung durch Compiler.

```
void print(int x) { ... }
void print(String s) { ... }
```

Überschreiben: abgeleitete Klasse definiert eine Methode mit gleicher Signatur neu. Entscheidung bei Objektmethoden dynamisch zur Laufzeit.

```
class A {
    void m() { IO.print("A"); }
```

```

}

class B extends A {
    @Override
    void m() { IO.print("B"); }
}

A x = new B();
x.m();           // B

```

Warum? Statischer Typ von `x` ist `A`, dynamischer Typ ist `B`. Bei dynamischer Bindung entscheidet der dynamische Typ.

3.4 `super` und Konstruktoren bei Vererbung

Ein Konstruktor der abgeleiteten Klasse muss die Basisklasse initialisieren. Das geschieht mit `super(...)`.

```

public class ColoredRectangle extends Rectangle {
    private Color color;

    public ColoredRectangle(int x, int y, int w, int h, Color color) {
        super(x, y, w, h);
        this.color = color;
    }
}

```

`super(...)` muss die erste Anweisung im Konstruktor sein. Mit `super.m()` kann eine überschriebene Basismethode aufgerufen werden.

```

@Override
public int hashCode() {
    return 31 * super.hashCode() + color.hashCode();
}

```

3.5 Polymorphismus: statischer und dynamischer Typ

Polymorphismus bedeutet: Eine Variable vom Typ einer Basisklasse kann Objekte der Basisklasse und aller abgeleiteten Klassen referenzieren.

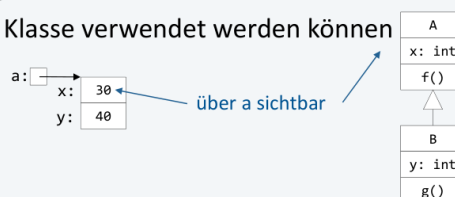
Statischer und dynamischer Datentyp



Statischer Typ (*static type*)

- wird in der Deklaration festgelegt und ist unveränderlich
- bestimmt, welche Elemente der Klasse verwendet werden können

```
A a;
```



Dynamischer Typ (*dynamic type*)

- ergibt sich durch die Zuweisung
- kann sich durch eine neuerliche Zuweisung wieder ändern
- bestimmt, welche Methode aufgerufen wird (dynamische Bindung)

```
a = new B(30 ↓ 40);
```

Statischer und dynamischer Datentyp. Quelle: 03 OOP, Folie 31.

```

Shape s;
s = new Circle();
s.draw();
s = new Rectangle();
s.draw();

```

Der statische Typ ist `Shape`. Er bestimmt, welche Methoden der Compiler erlaubt. Der dynamische Typ ist zur Laufzeit `Circle` oder `Rectangle`. Er bestimmt, welche überschreibende Methode tatsächlich ausgeführt wird.

```
Shape s = new Circle();
s.draw();           // erlaubt, wenn draw in Shape deklariert ist
s.radius();         // Compilerfehler, wenn radius nicht in Shape steht
```

3.6 Dynamische Bindung wirklich verstehen

Klausuraufgaben zur dynamischen Bindung sehen oft so aus:

```
class A {
    void m1() { IO.print("A"); }
    void m2() { this.m1(); }
    void m3() { m1(); }
}

class B extends A {
    @Override
    void m1() { IO.print("B"); }
}

A x = new B();
x.m2();
```

Ausgabe: B.

Warum? `x.m2()` ruft zwar die in A definierte Methode `m2` auf, aber innerhalb von `m2` ist `this` das tatsächliche B-Objekt. Der Aufruf `this.m1()` wird dynamisch gebunden und landet bei `B.m1()`.

Merksatz: Auch ein Methodenaufruf innerhalb einer Basisklassenmethode kann dynamisch an eine überschreibende Methode der abgeleiteten Klasse gebunden werden.

3.7 Klasse `Object`, `toString`, `equals`, `hashCode`

Alle Klassen erben letztlich von `Object`. Besonders relevant sind:

```
public String toString()
public boolean equals(Object obj)
public int hashCode()
```

`toString`

```
@Override
public String toString() {
    return "Color(r=%d,g=%d,b=%d)".formatted(red, green, blue);
}
```

`toString` wird oft automatisch verwendet, z. B. bei String-Verkettung.

`equals(Object o)`

Eine saubere `equals`-Methode prüft:

1. Ist es dasselbe Objekt?
2. Hat das andere Objekt überhaupt den passenden Typ?
3. Stimmen alle gleichheitsrelevanten Daten überein?

```
@Override
public boolean equals(Object o) {
    if (this == o) {
        return true;
    }
    if (!(o instanceof Color)) {
        return false;
    }
    Color other = (Color) o;
    return red == other.red
        && green == other.green
        && blue == other.blue;
}
```


Warum instanceof und Cast? Die Signatur lautet `equals(Object o)`, nicht `equals(Color o)`. Der Parameter kann also jedes Objekt oder `null` sein. Erst nach der Typprüfung darf man auf `red`, `green`, `blue` zugreifen.

hashCode

Wenn `equals` überschrieben wird, muss `hashCode` passend dazu überschrieben werden. Gleiche Objekte müssen denselben Hashcode haben.

```
@Override
public int hashCode() {
    int result = 17;
    result = 31 * result + red;
    result = 31 * result + green;
    result = 31 * result + blue;
    return result;
}
```

Die Multiplikation mit einer Primzahl wie 31 verteilt typische Werte besser als bloßes Addieren. Das wird in [Kapitel 7 Hashing](#) vertieft.

3.8 Abstrakte Klassen und Interfaces

Eine abstrakte Klasse kann nicht direkt instanziiert werden und kann abstrakte Methoden enthalten.

```
public abstract class Shape {
    public abstract void draw();
}
```

Eine abgeleitete konkrete Klasse muss die abstrakten Methoden implementieren:

```
public class Circle extends Shape {
    @Override
    public void draw() { ... }
}
```

Ein Interface beschreibt eine Schnittstelle. Klassen können mehrere Interfaces implementieren.

```
public interface Drawable {
    void draw();
}

public interface Movable {
    void moveBy(int dx, int dy);
}

public class Sprite implements Drawable, Movable {
    public void draw() { ... }
    public void moveBy(int dx, int dy) { ... }
}
```

Unterschied für die Klausur: Eine abstrakte Klasse kann gemeinsamen Zustand und gemeinsame Implementierung bündeln. Interfaces dienen vor allem der gemeinsamen Schnittstelle und erlauben eine Form von Mehrfachtypisierung.

3.9 Nachbilden von Mehrfachvererbung

Java erlaubt keine Mehrfachvererbung von Klassen:

```
class C extends A, B { }           // nicht erlaubt
```

Aber eine Klasse kann mehrere Interfaces implementieren:

```
class C extends A implements X, Y { ... }
```

Damit kann ein Objekt in mehreren Rollen auftreten:

```
Drawable d = new Sprite();
Movable m = new Sprite();
```

Das ist häufig die gewünschte Lösung, wenn mehrere unabhängige Fähigkeiten modelliert werden sollen.

3.10 Typtest und Typumwandlung

Wenn der statische Typ zu allgemein ist, benötigt man manchmal einen Typcheck und Cast.

```
Shape s = getShape();
if (s instanceof Circle) {
    Circle c = (Circle) s;
    c.radius();
}
```

Ohne instanceof kann ein falscher Cast zur ClassCastException führen. In `equals(Object o)` ist der Typcheck besonders wichtig, weil der Parameter laut Signatur jedes Objekt sein kann.

3.11 Mini-Check Kapitel 3

1. Was gibt folgender Code aus?

```
class A {
    void m() { IO.print("A"); }
    void n() { m(); }
}
class B extends A {
    void m() { IO.print("B"); }
}
A x = new B();
x.n();
```

Lösung: B.

2. Warum ist `boolean equals(Color c)` kein korrektes Überschreiben von `Object.equals`?

Lösung: Andere Signatur. Es ist Überladen, nicht Überschreiben. Korrekt ist `boolean equals(Object o)`.

Kapitel 4 - Datenorientierter Systementwurf und ATG

[Zurück zur Kapitelübersicht](#)

4.1 Klausurfokus

ATG ist laut Klausurinfo sicher klausurrelevant. Nicht erwartet wird eine vollständige Parser-/Interpreter-Implementierung. Erwartet wird dagegen:

- Grammatiknotation sicher lesen und schreiben,
- Terminalklassen wie `number` und `ident` beschreiben,
- LL(1)-Bedingung prüfen,
- nicht-LL(1)-konforme Regeln umformen,
- Attribute und semantische Aktionen passend einsetzen,
- einfache ATG-Aufgaben entwickeln.

4.2 Denkmodell: strukturierter Eingabestrom wird transformiert

Beim datenorientierten Systementwurf geht man von einem strukturierten Eingabedatenstrom aus. Zuerst wird seine Syntax beschrieben, dann seine Bedeutung und Verarbeitung.

Beispiele:

```
Hotel Mühlviertlerhof: ****
Anna: ****
Marco: *****
```

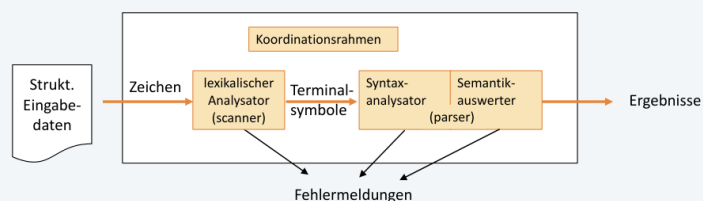
oder:

```
10 L (500,100,100)
20 C (700,300,100) (500,500,100)
E
```

Die typische Architektur besteht aus Scanner, Parser und Semantikauswerter.

Zum Denkmodell des datenorientierten Systementwurfs

Grobarchitektur



- Grundstruktur der Systemarchitektur bleibt immer gleich (Referenzarchitektur)
- Architekturmuster Pipes & Filters

Referenzarchitektur datenorientierter Systeme: Scanner, Parser, Semantikauswerter. Quelle: 04 ATG, Folie 8.

Für die Klausur reicht meistens: Syntax erkennen, Attribute definieren und semantische Aktionen an den richtigen Stellen eintragen.

4.3 Grammatikgrundlagen

Eine Grammatik beschreibt gültige Sätze einer Sprache.

```
Expr = Term { "+" Term }.
Term = Fact { "*" Fact }.
Fact = number | "(" Expr ")".
```

Wichtige Begriffe:

Begriff	Bedeutung
Terminalsymbol	direktes Symbol im Eingabetext, z. B. "+", "(", "cm"
Terminalklasse	Menge ähnlicher Terminals, z. B. number, ident, digit
Nichtterminal	syntaktische Kategorie, z. B. Expr, Term, Fact
Startsymbol	Nichtterminal, mit dem die Analyse beginnt
Alternative	Auswahl mit
Option	null- oder einmal mit [...]
Wiederholung	null- oder mehrmals mit { ... }

Punkt nicht vergessen: Regeln enden in der verwendeten Notation mit `.`

4.4 Terminalklassen beschreiben

Die Klausurinfo betont, dass Terminalklassen zumindest sprachlich oder mit Regex beschrieben werden können müssen.

Beispiele:

```
digit = "0" | "1" | ... | "9"
number = digit { digit }
ident = letter { letter | digit }
```

Als Regex:

```
number: [0-9]+
ident: [A-Za-z][A-Za-z0-9]*
```

Wenn cm, m und min als Einheiten vorkommen, sind das **Terminalsymbole**, keine Nichtterminale:

```
Unit = "cm" | "m" | "min".
```

4.5 LL(1)-Bedingung

Für rekursiven Abstieg muss der Parser anhand des nächsten Eingabesymbols entscheiden können, welche Alternative zu wählen ist.

LL(1)-Intuition: In jeder Alternative müssen unterschiedliche Anfangsterminalsymbole möglich sein.

Problematisch:

```
B = C | C "." C.
```

Beide Alternativen beginnen mit `C`. Mit nur einem Lookahead kann man nicht entscheiden, welche Alternative gemeint ist.

Umformung:

```
B = C [ "." C ].
```

Jetzt wird zuerst `C` erkannt. Danach entscheidet das nächste Symbol, ob noch `"." C` folgt.

Klausurstrategie: Gemeinsamen Anfang herausziehen.

4.6 Attributierte Grammatiken

Eine kontextfreie Grammatik beschreibt nur die Syntax. Eine attributierte Grammatik ergänzt Werte und Aktionen.

Notation aus den Folien:

```
Number ↑n = digit ↑d          ( . n := d . )
          { digit ↑d          ( . n := 10*n + d . ) }.
```

- Ausgangsattribute (↑) werden berechnet und nach außen geliefert.
- Eingangsattribute (↓) werden von außen in ein Nichtterminal hineingegeben.
- Semantische Aktionen stehen an der Stelle, an der ihre Werte verfügbar sind.

4.7 Beispiel: Anzahl von Bewertungen zählen

Aufgabenformat ähnlich der Beispielklausur:

```
Hotel Mühlviertlerhof: ****
Anna: ****
Marco: *****
Julia: ***
Thomas: *****
Herbert: *
```

Gesucht: Wie viele Gäste haben mindestens so viele Sterne vergeben wie das Hotel selbst?

Eine mögliche Grammatikstruktur:

```
Ratings ↑count =
  "Hotel" ident ":" Stars ↑hotelStars
  { GuestRating ↓hotelStars ↑ok }      ( . count := count + ok . )
  .

GuestRating ↓hotelStars ↑ok =
  ident ":" Stars ↑guestStars          ( . ok := guestStars >= hotelStars ? 1 : 0 . )
  .

Stars ↑n =
  "*"                                  ( . n := 1 . )
  { "*"                               ( . n := n + 1 . ) }
  .
```

Eine vollständig saubere ATG müsste die Initialisierung von count explizit setzen:

```
Ratings ↑count =
  "Hotel" ident ":" Stars ↑hotelStars ( . count := 0 . )
  { GuestRating ↓hotelStars ↑ok       ( . count := count + ok . ) }
  .
```

Verständnis: hotelStars wird nach unten weitergegeben, weil jede Gästebewertung diesen Vergleichswert benötigt. ok kommt aus der einzelnen Gästebewertung zurück.

4.8 Typische Fehler bei ATG-Aufgaben

1. **Terminalklassen vergessen:** number oder ident muss definiert oder beschrieben werden.
2. **Punkt am Ende der Regel vergessen.**
3. **LL(1)-Konflikt übersehen:** Alternativen beginnen gleich.
4. **Aktion zu früh:** Ein Wert wird verwendet, bevor er berechnet wurde.
5. **Eingangs-/Ausgangsattribute vertauscht:** Vergleichswert muss hinein, Ergebnis kommt heraus.
6. **Interpreter statt ATG:** Nicht unnötig Java-Parsercode schreiben, wenn eine Grammatik gefragt ist.

4.9 Mini-Check Kapitel 4

1. Warum ist folgende Regel nicht LL(1)-geeignet?

```
A = ident | ident "=" number.
```

2. Forme sie um.

Lösung:

```
A = ident [ "=" number ].
```

3. Beschreibe `number` als Terminalklasse.

Lösung: Eine nichtleere Folge von Ziffern, z. B. Regex `[0-9]+`.

Kapitel 5 - Ausnahmebehandlung

[Zurück zur Kapitelübersicht](#)

5.1 Klausurfokus

Ausnahmebehandlung ist klausurrelevant, aber oft in kleinen Tracing-Aufgaben. Man muss sicher wissen:

- Wann entsteht eine Exception?
- Welche `catch`-Klausel passt?
- Wird `finally` ausgeführt?
- Läuft das Programm danach weiter?
- Muss eine Exception mit `throws` deklariert werden?

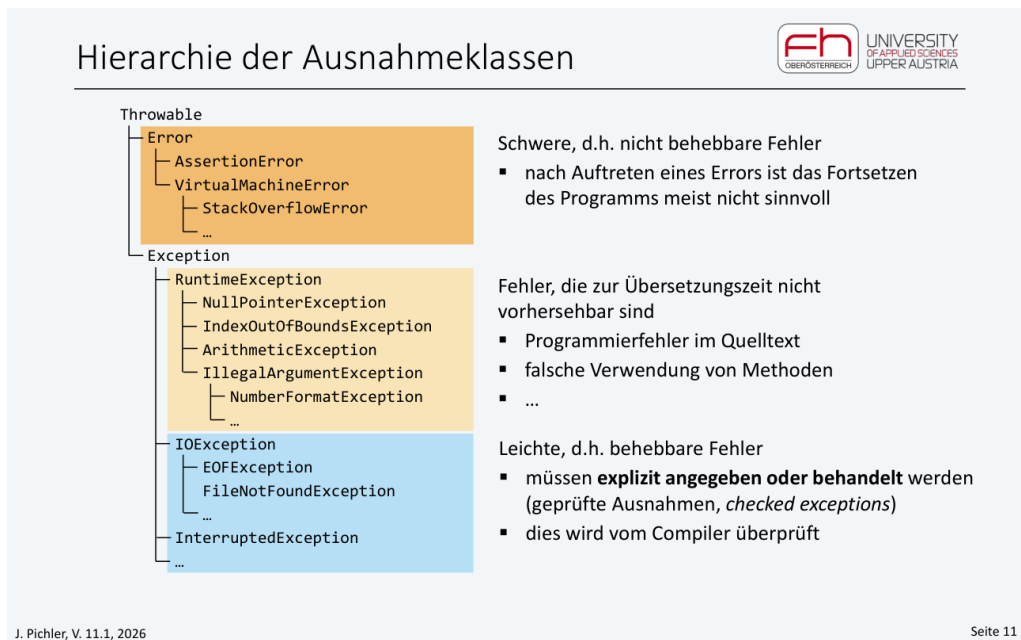
5.2 Warum Exceptions?

Ausnahmen treten auf, wenn während der Ausführung etwas Besonderes oder Fehlerhaftes passiert:

- Division durch 0,
- ungültiges Zahlenformat bei `Integer.parseInt`,
- Datei nicht gefunden,
- Zugriff auf `null`,
- Array-Index außerhalb der Grenzen.

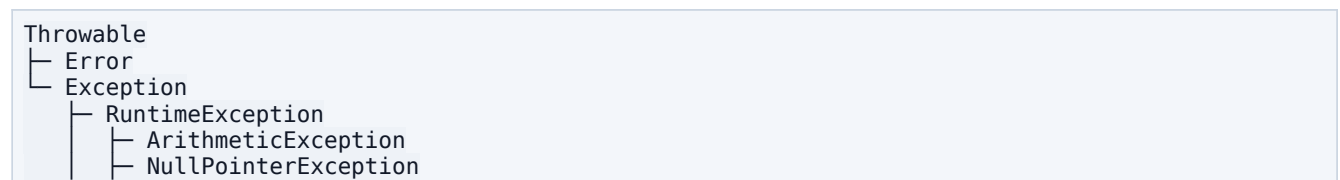
Statt undefiniert weiterzulaufen, erzeugt Java ein Exception-Objekt und wirft es. Wird es nicht behandelt, wird es an die Aufrufer weitergereicht. Wenn niemand reagiert, bricht das Programm ab.

5.3 Exception-Hierarchie



Exception-Hierarchie. Quelle: 05 Ausnahmebehandlung, Folie 11.

Grob:



```

└─ NumberFormatException
└─ IOException
└─ FileNotFoundException

```

- RuntimeException und Unterklassen sind unchecked.
- Viele andere Exception-Unterklassen sind checked und müssen behandelt oder deklariert werden.
- Error steht für schwere Fehler, die man normalerweise nicht behandelt.

5.4 try, catch, finally

Grundform:

```

try {
    // riskanter Code
} catch (NumberFormatException e) {
    // Behandlung
} catch (IOException e) {
    // Behandlung
} finally {
    // Abschlussarbeiten
}

```

Ablauf:

1. Der try-Block wird ausgeführt.
2. Wenn keine Exception entsteht, werden die catch-Blöcke übersprungen.
3. Wenn eine Exception entsteht, wird der erste passende catch-Block ausgeführt.
4. finally wird immer ausgeführt: ohne Exception, mit behandelter Exception und auch dann, wenn eine Exception weitergeworfen wird.
5. Nach finally geht es nur weiter, wenn keine unbehandelte Exception übrig ist.

5.5 Klausurtyp: Ausgabe bestimmen

Beispiel:

```

try {
    System.out.print("1");
    m();
    System.out.print("2");
} catch (NumberFormatException e) {
    System.out.print("3");
} catch (FileNotFoundException e) {
    System.out.print("4");
} finally {
    System.out.print("5");
}
System.out.print("6");

```

Fälle:

Fall	Ausgabe	Erklärung
m() wirft keine Exception	1256	try vollständig, kein catch, dann finally, dann weiter
m() wirft FileNotFoundException	1456	nach m() springt Ausführung in passenden catch, dann finally, dann weiter
m() wirft ArithmeticException	15 plus Programmabbruch	kein passender catch; finally läuft, danach wird Exception weitergeworfen; 6 wird nicht erreicht

Warum nicht 2? Sobald m() eine Exception wirft, wird der restliche try-Block übersprungen.

5.6 Checked vs. unchecked und throws

Unchecked Exceptions sind `RuntimeException` und Unterklassen. Sie müssen nicht in der Methodensignatur stehen.

```
public static int div(int a, int b) {
    return a / b;    // ArithmeticException möglich, aber kein throws nötig
}
```

Checked Exceptions müssen behandelt oder deklariert werden.

```
public static void readFile(String path) throws IOException {
    // Dateioperation, die IOException auslösen kann
}
```

Wenn eine Methode eine checked Exception nicht selbst abfängt, muss sie diese mit `throws` deklarieren.

5.7 throw und eigene Exceptions

Mit `throw` wird ein Exception-Objekt aktiv ausgelöst.

```
if (denom == 0) {
    throw new IllegalArgumentException("denom must not be 0");
}
```

Eigene Exceptions können definiert werden, wenn ein fachlicher Fehler präzise modelliert werden soll.

```
public class InvalidDigitException extends NumberFormatException {
    private final int position;

    public InvalidDigitException(int position) {
        super("invalid digit at position " + position);
        this.position = position;
    }

    public int getPosition() {
        return position;
    }
}
```

Für Klausuraufgaben reicht oft, vorhandene Exceptions sinnvoll zu verwenden, außer die Aufgabe verlangt explizit eine eigene Exception.

5.8 Eingabevalidierung bei Methoden

Die Klausurinfo sagt sinngemäß: Wenn Eingangsparameter validiert werden sollen, steht es oft ausdrücklich dabei. Trotzdem gilt für gute Lösungen:

- Public Methoden sollten ungültige Eingaben nicht stillschweigend akzeptieren.
- Private Methoden können stärker auf interne Vorbedingungen vertrauen.
- Bei klarer Invariante sollte der Konstruktor validieren.

Beispiel:

```
public Rational(int nom, int denom) {
    if (denom == 0) {
        throw new IllegalArgumentException("denom must not be 0");
    }
    this.nom = nom;
    this.denom = denom;
}
```

5.9 Mini-Check Kapitel 5

1. Wird `finally` ausgeführt, wenn im `try` eine nicht gefangene Exception entsteht?

Ja. Danach wird die Exception weitergereicht.

2. Muss `NumberFormatException` mit `throws` deklariert werden?

Nein. Sie ist eine `RuntimeException`.

3. Muss `IOException` mit `throws` deklariert oder gefangen werden?

Ja, wenn sie entstehen kann und nicht im selben Methodenrumpf behandelt wird.

Kapitel 6 - Pakete, Generics und Java-Bibliothek

[Zurück zur Kapitelübersicht](#)

6.1 Klausurfokus

Dieses Kapitel ist im Vergleich zu OOP, ATG, Hashing und Heap eher ein **Querschnittskapitel**. Es liefert aber wichtige Details, die in Codingaufgaben leicht Punkte kosten:

- Wo steht package und was bedeutet ein Paket als Sichtbarkeitsgrenze?
- Was ist der Unterschied zwischen `public`, `protected`, package visibility und `private`?
- Welche Importvarianten gibt es?
- Warum helfen Generics bei Typsicherheit?
- Welche Bibliotheksmethoden dürfen und sollten verwendet werden, besonders `Integer.parseInt`, `StringBuilder`, `Arrays.copyOf` und einfache Collections?

Nach `PRO_Klausurinfo.md` kommen Generics nicht bzw. nur minimal. Deshalb liegt der Schwerpunkt hier nicht auf komplizierten generischen Klassen, Wildcards oder Bounds, sondern auf sicherem Lesen und Verwenden einfacher generischer Container wie `List<Integer>`.

6.2 Pakete als Organisations- und Sichtbarkeitskonzept

Ein Paket ist eine Sammlung zusammengehöriger Klassen und Interfaces. In Java steht die Paketdeklaration in der ersten Zeile der Datei:

```
package printer.moves;

public class Moves {
    ...
}
```

Fehlt die package-Zeile, liegt die Klasse im namenlosen Paket. Für größere Programme ist das unüblich und für Klausuraufgaben meist nicht gemeint.

Pakete erfüllen zwei Hauptzwecke:

1. **Namensräume:** Zwei Klassen dürfen gleich heißen, wenn sie in unterschiedlichen Paketen liegen, z. B. `shapes.Circle` und `graphics.Circle`.
2. **Sichtbarkeitsgrenzen:** Was nicht `public` ist, ist außerhalb des Pakets nicht sichtbar.

Ein Paketname wird auf Verzeichnisse abgebildet:

```
printer/moves/Moves.java    -> package printer.moves;
printer/analyzer/Parser.java -> package printer.analyzer;
printer/Program.java        -> package printer;
```

Klausurverständnis: Ein Paket ist nicht nur „Ordnerstruktur“, sondern beeinflusst, welcher Code auf welche Klassen, Konstruktoren, Methoden und Datenkomponenten zugreifen darf.

6.3 Sichtbarkeit: public, protected, package, private

Die Sichtbarkeit ist prüfungsrelevant, weil sie mit dem Geheimnisprinzip aus [Kapitel 2](#) und Vererbung aus [Kapitel 3](#) zusammenhängt.

Sichtbarkeitsattribute



Sichtbarkeit von Klassen

- `public` Klassen können auch außerhalb des Pakets benutzt werden
- nicht `public` Klassen sind nur innerhalb des Pakets sichtbar

Sichtbarkeit von Datenkomponenten und Methoden

- wird durch `public`, `protected` oder `private` definiert
- ohne Angabe eines Sichtbarkeitsattributs gilt der Sichtbarkeitsbereich Paket (*package visibility*)

Datenk./Methoden sind zugreifbar in ...	... wenn für Datenk./Methoden gilt:			
	<code>public</code>	<code>protected</code>	(<i>package</i>)	<code>private</code>
... selber Klasse	ja	ja	ja	ja
... anderer Klasse in gleichem Paket	ja	ja	ja	nein
... abgeleit. Klasse in anderem Paket	ja	ja	nein	nein
... anderer Klasse in anderem Paket	ja	nein	nein	nein

J. Pichler, V. 11.1, 2026

Seite 7

Sichtbarkeit von Klassen, Datenkomponenten und Methoden. Quelle: 06 Java-Bibliothek, Folie 7.

Kurzfassung für die Klausur:

- `public`: überall sichtbar.
- `protected`: in derselben Klasse, im selben Paket und in abgeleiteten Klassen auch aus anderen Paketen sichtbar.
- Kein Modifikator / `package visibility`: in derselben Klasse und im selben Paket sichtbar, aber nicht außerhalb des Pakets.
- `private`: nur innerhalb derselben Klasse sichtbar.

Die Tabelle in der Folie zeigt dieselben Regeln vollständig; für Aufgaben reicht es meist, diese vier Fälle sicher unterscheiden zu können.

Wichtig ist die Unterscheidung zwischen **Klasse** und **Objekt**: `private` ist klassenbezogen. Eine Methode derselben Klasse darf auf `private` Datenkomponenten eines anderen Objekts derselben Klasse zugreifen.

```
public class Rational {
    private int nom;
    private int denom;

    public boolean sameDenom(Rational other) {
        return this.denom == other.denom; // erlaubt: gleiche Klasse
    }
}
```

Das ist in Klausuraufgaben zu `equals` oder Vergleichsmethoden häufig relevant.

6.4 Importvarianten und Qualifikation

Eine `public` Klasse aus einem anderen Paket kann auf drei Arten benutzt werden.

```
import shapes.Circle;

class App {
    Circle c;
}

import shapes.*;

class App {
    Circle c;
    Rectangle r;
}

class App {
    shapes.Circle c1;
```

```
graphics.Circle c2;
}
```

import shapes.*; importiert die öffentlichen Klassen des Pakets shapes, aber **nicht automatisch Unterpakete**.
import java.util.*; importiert also nicht java.util.concurrent.*.

Klausurtyp: Wenn zwei Klassen denselben einfachen Namen haben, ist die voll qualifizierte Schreibweise eindeutig:

```
shapes.Circle c1;
graphics.Circle c2;
```

6.5 Generics: nur das klausurrelevante Minimum

Ohne Generics weiß der Compiler bei allgemeinen Containern oft nur: Das Element ist ein `Object`.

```
ArrayList list = new ArrayList();
list.add(new Circle(...));
list.add(new Person(...));

Shape s = (Shape) list.get(1); // kann zur Laufzeit ClassCastException auslösen
```

Mit Generics wird der zulässige Elementtyp zur Compilezeit eingeschränkt:

```
ArrayList<Shape> shapes = new ArrayList<>();
shapes.add(new Circle(...));
shapes.add(new Line(...));
// shapes.add(new Person(...)); // Compilerfehler, falls Person kein Shape ist

Shape s = shapes.get(0); // kein Cast nötig
```

Kernideen:

- E, T, K, V sind Typparameter, also Platzhalter für konkrete Typen.
- `ArrayList<String>` ist für den Compiler ein anderer Typ als `ArrayList<Integer>`.
- Typparameter verbessern Typsicherheit, weil Fehler früher erkannt werden.
- Zur Laufzeit werden Typparameter durch **type erasure** nicht als echte Typinformation gespeichert.
- Deshalb ist `new E[16]` nicht möglich; oft sieht man intern `new Object[16]` mit Cast.

Für die PRO2-Klausur reicht normalerweise, generische Bibliotheksklassen lesen und einfach verwenden zu können:

```
List<Integer> adj = new LinkedList<>();
adj.add(3);
for (int w : adj) {
    IO.println(w);
}
```

Das wird besonders in [Kapitel 9](#) bei Adjazenzlisten gebraucht.

6.6 Bibliothek: wann verwenden, wann selbst implementieren?

Die Java-Bibliothek enthält sehr viele fertige Klassen und Methoden. In einer Klausur ist aber entscheidend, was die Aufgabenstellung erlaubt. `PRO_Klausurinfo.md` nennt ausdrücklich `Integer.parseInt` als Methode, die man kennen und verwenden darf.

Wichtige Beispiele:

```
int x = Integer.parseInt("42"); // String -> int
String s = String.valueOf(42); // int -> String

StringBuilder sb = new StringBuilder();
sb.append("A");
sb.append(',');
sb.append("B");
String result = sb.toString();

int[] a = {1, 2, 3};
int[] b = Arrays.copyOf(a, 2 * a.length);
```

Wann ist `StringBuilder` sinnvoll? Wenn in einer Schleife viele Zeichenketten zusammengesetzt werden, vermeidet `StringBuilder` viele Zwischenobjekte. Eine `join`-Aufgabe aus der Beispielklausur kann man damit sauber lösen:

```
public static String join(String[] text, char delimiter) {
    StringBuilder sb = new StringBuilder();
    boolean first = true;
    for (String s : text) {
        if (s != null) {
            if (!first) {
                sb.append(delimiter);
            }
            sb.append(s);
            first = false;
        }
    }
    return sb.toString();
}
```

Der entscheidende Denkpunkt ist nicht `StringBuilder` selbst, sondern die korrekte Behandlung von `null` und Trennzeichen.

6.7 Mini-Check Kapitel 6

1. Ist eine Klasse ohne `public` außerhalb ihres Pakets sichtbar?

Nein. Sie hat package visibility.

2. Importiert `import shapes.*`; automatisch Unterpakete?

Nein. Nur öffentliche Typen direkt aus `shapes`.

3. Warum sind Generics nützlich?

Sie verschieben viele Typfehler von der Laufzeit in die Compilezeit und reduzieren Casts.

4. Warum ist `new E[10]` in einer generischen Klasse problematisch?

Weil Typparameter zur Laufzeit nicht als konkrete Array-Komponententypen verfügbar sind.

Kapitel 7 - Hashing-basierte Suche

[Zurück zur Kapitelübersicht](#)

7.1 Klausurfokus

Hashing ist prüfungsrelevant, aber wahrscheinlich eher als Verständnis- und kleine Implementierungsaufgabe.

Nach `PRO_Klausurinfo.md` sind besonders wichtig:

- warum Kollisionen nicht vollständig vermeidbar sind,
- welche Möglichkeiten zur Kollisionsbehandlung existieren,
- wann Hashing annähernd $O(1)$ erreicht,
- wann eine Hashfunktion gut ist,
- wie `equals` und `hashCode` bei eigenen Klassen zusammenhängen.

Das Thema verbindet Datenstrukturen mit OOP: Wer `equals` überschreibt, muss auch `hashCode` passend überschreiben, wenn Objekte in hashbasierten Datenstrukturen korrekt funktionieren sollen.

7.2 Grundidee: direkter Zugriff über berechneten Index

Bei sequenzieller Suche dauert das Finden im schlechtesten Fall proportional zur Anzahl der Elemente: $O(n)$. Binäre Suche ist besser, braucht aber sortierte Daten und dauert $O(\log n)$.

Hashing versucht, den Speicherplatz eines Elements direkt aus seinem Schlüssel zu berechnen:

```
Schlüssel -> Hashcode -> Tabellenindex
```

In Java gibt eine Hashfunktion typischerweise eine ganze Zahl zurück:

```
int h = key.hashCode();
int i = Math.floorMod(h, table.length);
```

`Math.floorMod` ist hier besser als `h % table.length`, weil `hashCode()` negativ sein kann. In einer Klausur kann auch eine einfachere positive Hashfunktion vorgegeben sein.

Begriff: Eine Hashtabelle ist ein Feld fester oder dynamisch wachsender Größe `m`. Das Element mit Schlüssel `key` wird an oder nahe bei `hash(key)` gespeichert.

7.3 Warum Kollisionen unvermeidbar sind

Eine Kollision liegt vor, wenn zwei verschiedene Schlüssel auf denselben Hashcode oder zumindest denselben Tabellenindex abgebildet werden.

```
H(k1) = H(k2), obwohl k1 != k2
```

Kollisionen sind prinzipiell unvermeidbar, sobald die Menge möglicher Schlüssel größer ist als die Zahl der Tabellenplätze. Es gibt unendlich viele mögliche Strings, aber nur endlich viele Tabellenindizes.

Prüfungsantwort: Eine gute Hashfunktion kann Kollisionen selten machen, aber nicht garantieren, dass sie nie auftreten.

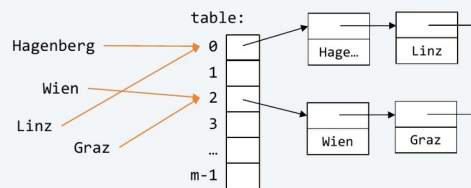
7.4 Kollisionsbehandlung durch Verkettung

Bei der Verkettung enthält jedes Tabellenfeld den Kopf einer Liste. Alle Elemente, deren Schlüssel auf denselben Index fallen, werden in dieser Liste gespeichert.

Kollisionsbehandlung: Verkettung von Kollisionen



Datenobjekte mit gleichem Hashcode werden in einer einfach-verketteten Liste gespeichert



Zusätzliche Komponente next erforderlich

```
class Node {
    String key;
    String data;
    Node next;

    Node(String key, String data) {
        this.key = key;
        this.data = data;
        this.next = null;
    }
}
```

J. Pichler, V. 11.1, 2026

Seite 8

Kollisionsbehandlung durch Verkettung. Quelle: 07 Hashing, Folie 8.

Typische Knotenstruktur:

```
class Node {
    String key;
    String data;
    Node next;

    Node(String key, String data) {
        this.key = key;
        this.data = data;
        this.next = null;
    }
}
```

Einfügen ohne Duplikate:

```
public void put(String key, String data) {
    if (key == null) {
        throw new IllegalArgumentException("key must not be null");
    }
    int i = hashCode(key);
    Node n = table[i];
    while (n != null && !n.key.equals(key)) {
        n = n.next;
    }
    if (n == null) {
        Node newNode = new Node(key, data);
        newNode.next = table[i];
        table[i] = newNode;
        size++;
    } else {
        n.data = data;
    }
}
```

Suchen:

```
public String get(String key) {
    if (key == null) {
        throw new IllegalArgumentException("key must not be null");
    }
    Node n = table[hashCode(key)];
    while (n != null && !n.key.equals(key)) {
        n = n.next;
    }
    return (n == null) ? null : n.data;
}
```


Tiefe Idee: Die Suche ist nicht „magisch konstant“. Sie ist nur dann schnell, wenn die Listen kurz bleiben. Die durchschnittliche Listenlänge ist ungefähr n / m , also Anzahl gespeicherter Elemente geteilt durch Tabellengröße.

7.5 Kollisionsbehandlung durch offene Adressierung

Bei offener Adressierung werden auch Kollisionsobjekte direkt in der Tabelle gespeichert. Ist der berechnete Platz belegt, sucht man nach einem Ersatzplatz.

Kollisionsbehandlung: Offene Adressierung



- Datensätze auch bei Kollisionen in Tabelle speichern (für $n < m$)
- Verschiedene Strategien um "Ersatzposition" zu finden

Lineare Kollisionsstrategie

- Wenn Position belegt ist, wird erste freie Position dahinter gesucht

```
private int keyPos(String key) {
    int i = hashCode(key);
    while (table[i] != null && !table[i].key.equals(key)) {
        i = (i + 1) % table.length;
    }
    return i;
}
```

Hier ohne Prüfung, ob überhaupt noch eine Position frei ist!

J. Pichler, V. 11.1, 2026

Seite 15

Lineare offene Adressierung. Quelle: 07 Hashing, Folie 15.

Lineares Sondieren:

```
private int keyPos(String key) {
    int i = hashCode(key);
    while (table[i] != null && !table[i].key.equals(key)) {
        i = (i + 1) % table.length;
    }
    return i;
}
```

Einfügen:

```
public void put(String key, String data) {
    int i = keyPos(key);
    if (table[i] == null) {
        size++;
    }
    table[i] = new Node(key, data);
}
```

Suchen:

```
public String get(String key) {
    int i = keyPos(key);
    return (table[i] == null) ? null : table[i].data;
}
```

Bei offener Adressierung muss die Tabelle genügend freie Plätze haben. Außerdem ist Entfernen schwieriger als bei Verkettung, weil eine einfach auf null gesetzte Lücke spätere Suchketten abbrechen könnte. Man braucht dann z. B. Tombstones oder Rehashing.

7.6 Lastfaktor und wann $0(1)$ realistisch ist

Der Lastfaktor ist:

$$\alpha = n / m$$

- n : Anzahl gespeicherter Elemente,
- m : Größe der Hashtabelle.

Bei Verkettung entspricht α ungefähr der durchschnittlichen Listenlänge. Bei offener Adressierung beschreibt α , wie voll die Tabelle ist.

Hashing erreicht erwartetes $O(1)$ nur unter Bedingungen:

1. Die Hashfunktion verteilt Schlüssel ungefähr gleichmäßig.
2. Der Lastfaktor bleibt kontrolliert, z. B. durch Vergrößern der Tabelle.
3. Gleichheit wird korrekt geprüft.

Wird die Tabelle zu voll, steigt die Zahl der Vergleiche. Dann ist Hashing nicht mehr praktisch konstant.

7.7 Gute Hashfunktionen

Eine gute Hashfunktion sollte:

- schnell berechenbar sein,
- die verwendeten Datenkomponenten berücksichtigen,
- die Schlüssel möglichst gleichmäßig über die Tabelle verteilen,
- gleiche Objekte nach `equals` immer auf denselben Hashcode abbilden.

Sehr einfache, aber oft ausreichende Struktur für eigene Klassen:

```
@Override
public int hashCode() {
    int h = 17;
    h = 31 * h + x;
    h = 31 * h + y;
    h = 31 * h + color;
    return h;
}
```

Die Multiplikation mit einer kleinen Primzahl wie 31 hilft, Reihenfolge und Positionen der Datenkomponenten besser einzumischen als bloßes Addieren.

7.8 Zusammenhang von `equals` und `hashCode`

Die wichtigste Regel:

Wenn `a.equals(b)` `true` ist, dann muss `a.hashCode() == b.hashCode()` gelten.

Die Umkehrung muss nicht gelten. Zwei ungleiche Objekte dürfen denselben Hashcode haben; das ist nur eine Kollision.

Beispiel:

```
public class Point {
    private int x;
    private int y;

    @Override
    public boolean equals(Object obj) {
        if (obj instanceof Point) {
            Point p = (Point) obj;
            return x == p.x && y == p.y;
        }
        return false;
    }

    @Override
    public int hashCode() {
        int h = 17;
        h = 31 * h + x;
        h = 31 * h + y;
        return h;
    }
}
```

```
}  
}
```

Warum ist das klausurwichtig? Wenn `equals` zwei Punkte mit gleichen Koordinaten als gleich betrachtet, `hashCode` aber die Koordinaten ignoriert oder inkonsistent ist, funktionieren `HashSet`/`HashMap`-ähnliche Strukturen nicht zuverlässig.

Mehr zur korrekten `equals`-Signatur steht in [Kapitel 3](#).

7.9 Mini-Check Kapitel 7

1. Warum kann man Kollisionen nicht vollständig vermeiden?

Weil viele mögliche Schlüssel auf endlich viele Tabellenplätze abgebildet werden.

2. Was ist der Unterschied zwischen Verkettung und offener Adressierung?

Bei Verkettung liegen Kollisionsobjekte in Listen außerhalb bzw. an Tabellenfeldern. Bei offener Adressierung liegen alle Objekte in der Tabelle selbst.

3. Wann ist Hashing erwartbar $O(1)$?

Bei guter Verteilung und kontrolliertem Lastfaktor.

4. Was muss gelten, wenn `a.equals(b)` wahr ist?

Dann müssen beide denselben Hashcode liefern.

Kapitel 8 - Heap-Datenstruktur

[Zurück zur Kapitelübersicht](#)

8.1 Klausurfokus

Nach PRO_Klausurinfo.md ist **Heapsort nicht prüfungsrelevant**. Relevant ist die Heap-Datenstruktur selbst:

- Priority Queue verstehen,
- Baum zu Feld und Feld zu Baum zeichnen,
- Position 0 im Feld freilassen,
- Heap-Eigenschaft prüfen,
- verletzte Heap-Ordnung mit `swim` oder `sink` wiederherstellen,
- `insert` und `removeMax` erklären oder teilweise implementieren.

Die Beispielklausur enthält genau eine Aufgabe in diesem Stil: Feld-Baum-Abbildung und Wiederherstellung der Heap-Ordnung.

8.2 Priority Queue als Motivation

Ein Stack entfernt immer das zuletzt eingefügte Element (LIFO). Eine Queue entfernt immer das zuerst eingefügte Element (FIFO). Eine Priority Queue entfernt dagegen das Element mit höchster Priorität, bei einem Max-Heap also das größte Element.

Operationen:

<code>insert(e)</code>	fügt ein Element ein
<code>removeMax()</code>	entfernt und liefert das größte Element

Ein unsortiertes Feld kann `insert` in $O(1)$, aber `removeMax` braucht $O(n)$. Ein sortiertes Feld kann `removeMax` in $O(1)$, aber `insert` braucht $O(n)$. Der Heap erreicht für beide Operationen $O(\log n)$.

8.3 Binärer Heap als kompletter Binärbaum

Ein Max-Heap ist ein kompletter Binärbaum mit Heap-Ordnung:

Jeder Knoten ist mindestens so groß wie seine Kinder.

Daraus folgt:

- Das größte Element steht an der Wurzel.
- Die Kinder untereinander müssen nicht sortiert sein.
- Ein Heap ist keine vollständig sortierte Struktur.
- Suchen nach einem beliebigen Element ist keine typische Heap-Operation.

Ein kompletter Binärbaum ist bis auf die letzte Ebene vollständig gefüllt. In der letzten Ebene dürfen nur rechts Knoten fehlen. Deshalb ist die Höhe ungefähr $\log_2(n)$.

8.4 Feldrepräsentation: Position 0 bleibt frei

Ein kompletter Binärbaum lässt sich sehr elegant in einem Feld speichern. In den Folien wird Position 0 freigelassen. Die Wurzel steht bei Index 1.

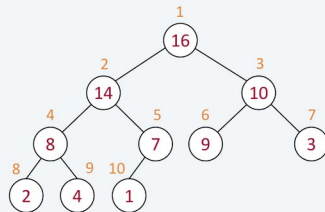
Heap in der Realisierungsform Feld

Ein kompletter Binärbaum kann auch **in Form eines Felds** realisiert werden.

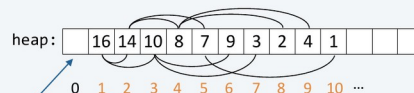
Die Werte in einem Feld `heap[1:n]` bilden einen Heap, wenn `heap[1]` die Wurzel ist und für jedes Element an der Stelle i gilt, dass

- sich sein linker Nachfolger an der Position $2i$ befindet
- sich sein rechter Nachfolger an der Position $2i + 1$ befindet
- sein Vorgänger an der Position $i/2$ zu finden ist

Beispiel



$\text{Left}(i) = 2i$
 $\text{Right}(i) = 2i + 1$
 $\text{Parent}(i) = i / 2$
 für $i > 1$: $\text{heap}[\text{Parent}(i)] \geq \text{heap}[i]$



Position 0 wird nicht genutzt

J. Pichler, V. 11.1, 2026

Seite 9

Feld-Baum-Abbildung eines Heaps. Quelle: 08 Heap, Folie 9.

Für einen Knoten an Position i gilt:

```

left(i)  = 2 * i
right(i) = 2 * i + 1
parent(i) = i / 2      // ganzzahlige Division
  
```

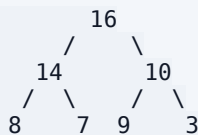
Heap-Eigenschaft im Feld:

für $i > 1$: $\text{heap}[i / 2] \geq \text{heap}[i]$

Typische Klausuraufgabe: Gegeben ist:

Index:	0	1	2	3	4	5	6	7
heap:	-	16	14	10	8	7	9	3

Der Baum ist:



8.5 swim: ein zu großes Element steigt auf

`swim` wird benötigt, wenn ein Element größer als sein Vorgänger ist. Das passiert typischerweise nach `insert`, weil das neue Element zuerst am Ende eingefügt wird.

```

private void swim(int k) {
    while (k > 1 && less(k / 2, k)) {
        swap(k / 2, k);
        k = k / 2;
    }
}
  
```

Bedeutung:

1. Solange k nicht die Wurzel ist und der Parent kleiner ist als $\text{heap}[k]$, wird getauscht.
2. Danach wandert k auf die Parent-Position.
3. Am Ende ist die Heap-Ordnung auf diesem Pfad wiederhergestellt.

Nicht verwechseln: `swim` vergleicht immer mit dem Parent, nicht mit den Kindern.

8.6 Einfügen mit `insert`

Einfügen besteht aus zwei Schritten:

1. Neues Element an die erste freie Position am Ende setzen.
2. Mit `swim` nach oben wandern lassen, solange die Heap-Ordnung verletzt ist.

```
public void insert(int e) {
    n++;
    heap[n] = e;
    swim(n);
}
```

Wenn das Feld voll ist, muss eine robuste Implementierung vorher vergrößern. Für eine Klausur kann die Aufgabe aber auch annehmen, dass genügend Platz vorhanden ist.

8.7 `sink`: ein zu kleines Element sinkt ab

`sink` wird benötigt, wenn ein Element kleiner als eines seiner Kinder ist. Dann wird es mit dem größeren Kind vertauscht, weil nur so die Max-Heap-Eigenschaft wiederhergestellt wird.

```
private void sink(int k) {
    while (2 * k <= n) {
        int j = 2 * k; // linkes Kind
        if (j < n && less(j, j + 1)) { // rechtes Kind ist größer
            j++;
        }
        if (!less(k, j)) {
            break; // Heap-Ordnung stimmt
        }
        swap(k, j);
        k = j;
    }
}
```

Wichtige Detailpunkte:

- `2 * k <= n` bedeutet: Es gibt mindestens ein linkes Kind.
- `j < n` prüft, ob auch ein rechtes Kind existiert.
- Es wird mit dem größeren Kind verglichen, nicht einfach immer mit links.
- Wenn `heap[k] >= heap[j]`, ist die Position gefunden.

8.8 Entfernen des Maximums mit `removeMax`

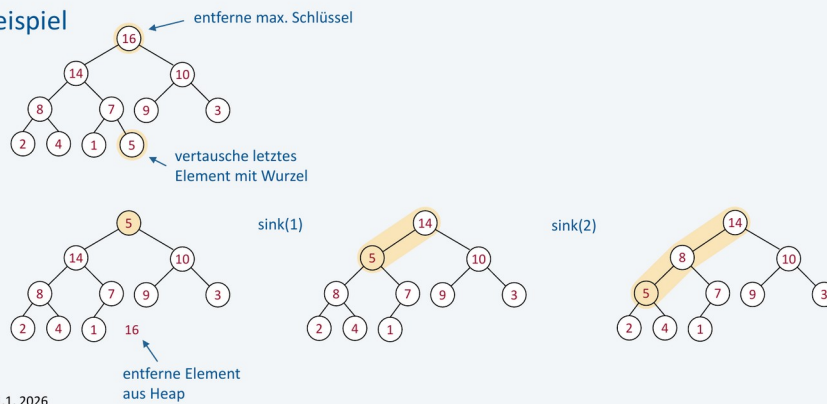
Das Maximum steht bei einem Max-Heap immer an der Wurzel `heap[1]`. Nach dem Entfernen muss der Baum weiterhin komplett bleiben. Deshalb wird nicht einfach die Wurzel gelöscht, sondern das letzte Element an die Wurzel gesetzt und dann abgesenkt.

Binärer Heap: entferne das Maximum

Lösungsidee

- vertausche Schlüssel in der Wurzel mit dem Element am Ende (Position n)
- nutze Algorithmus `sink`, um Heap-Ordnung wieder herzustellen

Beispiel



J. Pichler, V. 11.1, 2026

Seite 14

`removeMax`: Wurzel mit letztem Element tauschen und danach `sink`. Quelle: 08 Heap, Folie 14.

```
public int removeMax() {
    if (n < 1) {
        throw new IllegalStateException("empty heap");
    }
    int max = heap[1];
    swap(1, n);
    n--;
    sink(1);
    return max;
}
```

Warum zuerst tauschen? Das letzte Element lässt sich entfernen, ohne die Kompaktheit des Baums zu zerstören. Die Wurzelposition wird dann mit `sink` repariert.

8.9 Laufzeitverständnis

Ein kompletter Binärbaum mit n Knoten hat Höhe $O(\log n)$. `swim` und `sink` bewegen ein Element entlang eines Pfads von unten nach oben oder von oben nach unten. Deshalb gilt:

Operation	Laufzeit
<code>insert</code>	$O(\log n)$
<code>removeMax</code>	$O(\log n)$
<code>size</code>	$O(1)$
max/Wurzel lesen	$O(1)$
beliebiges Element suchen	nicht effizient, typischerweise $O(n)$

8.10 Typische Heap-Zeichenaufgabe

Gegeben:

Index:	0	1	2	3	4	5	6
heap:	-	20	18	10	9	30	7

Prüfung:

- `heap[1] = 20`, Kinder 18 und 10 -> ok.
- `heap[2] = 18`, Kinder 9 und 30 -> verletzt, weil $30 > 18$.

Reparatur: 30 muss mit 18 getauscht werden; danach steht 30 bei Index 2. Nun ist aber $30 > 20$, also muss nochmals mit der Wurzel getauscht werden. Das ist ein `swim` von Position 5.

Ergebnis:

Index:	0	1	2	3	4	5	6
heap:	-	30	20	10	9	18	7

8.11 Mini-Check Kapitel 8

1. Warum bleibt Feldposition 0 frei?

Damit Parent/Child-Formeln besonders einfach sind: Parent $i/2$, Kinder $2i$ und $2i+1$.

2. Ist ein Heap vollständig sortiert?

Nein. Er garantiert nur Parent $\geq$ Kinder.

3. Wann verwendet man `swim`?

Wenn ein Element größer als sein Parent ist, typischerweise nach dem Einfügen.

4. Wann verwendet man `sink`?

Wenn ein Element kleiner als mindestens eines seiner Kinder ist, typischerweise nach `removeMax`.

Kapitel 9 - Graphen

[Zurück zur Kapitelübersicht](#)

9.1 Klausurfokus

Graphen sind klausurrelevant vor allem in Form grundlegender Begriffe und einfacher Java-Methoden auf Adjazenzlisten. Laut `PRO_Klausurinfo.md` könnte etwa eine Methode zur Adjazenzlistenrepräsentation drankommen.

Wichtig sind:

- Knoten, Kanten, gerichtete und ungerichtete Graphen,
- gewichtete Graphen,
- Grad, Eingangsgrad, Ausgangsgrad,
- Weg, einfacher Weg, Zyklus, Zusammenhang,
- Adjazenzmatrix vs. Adjazenzliste,
- Interface `Graph/Digraph`, Klasse `AdjListDigraph`,
- Methoden `addEdge`, `removeEdge`, `outdegree`, `indegree`, `degree`, `reverse`.

9.2 Grundbegriffe

Ein Graph besteht aus einer Menge von Knoten und einer Menge von Kanten:

$$G = (V, E)$$

- V ist die Menge der vertices/Knoten.
- E ist die Menge der edges/Kanten.

Bei einem **ungerichteten Graphen** verbindet eine Kante zwei Knoten ohne Richtung. Bei einem **gerichteten Graphen** ist eine Kante ein geordnetes Paar (v, w) und zeigt von v nach w .

Ein **gewichteter Graph** ordnet jeder Kante einen Wert zu, etwa Entfernung, Kosten oder Dauer.

Beispiele:

- Straßenkarte ohne Einbahnstraßen: ungerichteter Graph.
- Produktionsschritte mit Abhängigkeiten: gerichteter Graph.
- Routenplanung mit Entfernungen: gewichteter Graph.

9.3 Wege, Zyklen und Zusammenhang

Ein Weg ist eine Folge von Knoten, bei der aufeinanderfolgende Knoten durch Kanten verbunden sind:

$$v_1, v_2, \dots, v_n$$

Ein Weg ist **einfach**, wenn kein Knoten mehrfach vorkommt. Die Weglänge ist die Anzahl der Kanten, nicht die Anzahl der Knoten.

Ein Zyklus ist ein Weg, bei dem Anfangs- und Endknoten gleich sind.

Ein Graph heißt **zusammenhängend**, wenn zwischen jedem Paar von Knoten ein Weg existiert. Bei gerichteten Graphen muss man genauer unterscheiden, ob Wege in Richtung der Kanten gemeint sind; für PRO2 genügt meist das grundlegende Verständnis.

9.4 Grade bei gerichteten Graphen

Bei gerichteten Graphen unterscheidet man:

```

OutDeg(v) = Anzahl der von v ausgehenden Kanten
InDeg(v)  = Anzahl der in v endenden Kanten
Deg(v)    = InDeg(v) + OutDeg(v)

```

Spezielle Knoten:

- Quelle: Eingangsgrad 0.
- Senke: Ausgangsgrad 0.
- Isolierter Knoten: Eingangsgrad und Ausgangsgrad 0.

Klausurfehler: Bei einem gerichteten Graphen ist (v, w) nicht dasselbe wie (w, v) .

9.5 Repräsentation: Knoten als Nummern

In Implementierungen werden Knoten häufig als ganze Zahlen von 0 bis $n - 1$ repräsentiert.

Zusätzliche Informationen liegen separat:

```

String[] names = new String[n];
Person[] persons = new Person[n];

```

Der Graph selbst speichert dann nur die Adjazenzbeziehungen.

9.6 Adjazenzmatrix vs. Adjazenzliste

Adjazenzmatrix:

```
boolean[][] adj = new boolean[n][n];
```

- $adj[v][w] == \text{true}$ bedeutet: Kante von v nach w existiert.
- Vorteil: Kantenabfrage in $O(1)$.
- Nachteil: Speicherbedarf $O(n^2)$, auch wenn es nur wenige Kanten gibt.

Adjazenzliste:

```
List<Integer>[] adj;
```

- $adj[v]$ enthält alle Nachfolger von v .
- Vorteil: Speicherbedarf ungefähr $O(|V| + |E|)$ bzw. für Kantenlisten $O(|E|)$.
- Nachteil: Test, ob eine bestimmte Kante existiert, braucht Suche in einer Liste.

Adjazenzliste

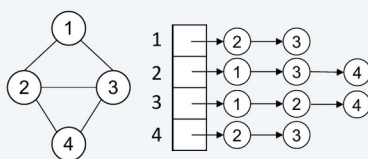


Adjazenzbeziehungen werden in einem Feld der Länge $n = |V|$ repräsentiert:

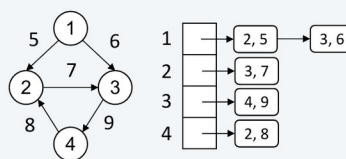
- **Index** ist Knotennummer; in jedem Feldelement ankert Liste aller Knoten, die mit dem Indexknoten verbunden sind
- **Listenelemente** enthalten Knotennummer und bei gewichteten Graphen numerischen Wert

Vorteil: geringer Speicherbedarf, $O(|E|)$ Nachteil: langsamer Zugriff, $O(|E|/n)$

Beispiele:



Ungewichteter ungerichteter Graph



Gewichteter gerichteter Graph

Adjazenzlisten speichern pro Knoten die angrenzenden Knoten. Quelle: 09 Graphen, Folie 10.

Wann welche Darstellung?

- Dichte Graphen mit vielen Kanten: Matrix kann sinnvoll sein.
- Dünne Graphen mit wenigen Kanten: Adjazenzliste spart Speicher.

9.7 Interfaces `Graph` und `Digraph`

Die Folien verwenden sinngemäß diese Schnittstellen:

```
public interface Graph {
    int getNrOfVertices();
    int getNrOfEdges();
    void addEdge(int v, int w);
    void removeEdge(int v, int w);
    Iterable<Integer> getAdj(int v);
    int degree(int v);
}

public interface Digraph extends Graph {
    int outdegree(int v);
    int indegree(int v);
    Digraph reverse();
}
```

`Iterable<Integer>` erlaubt die Verwendung in einer for-each-Schleife:

```
for (int w : graph.getAdj(v)) {
    IO.println(w);
}
```

Das ist ein gutes Beispiel für minimale Generics aus [Kapitel 6](#).

9.8 Klasse `AdjListDigraph`

Eine typische Repräsentation für einen gerichteten Graphen:

```
public class AdjListDigraph implements Digraph {
    private int nrOfVertices;
    private int nrOfEdges;
    private List<Integer>[] adj;
    private int[] indegree;

    public AdjListDigraph(int nrOfVertices) {
        this.nrOfVertices = nrOfVertices;
        this.nrOfEdges = 0;
        this.adj = (List<Integer>[]) new List<?>[nrOfVertices];
        for (int v = 0; v < nrOfVertices; v++) {
            this.adj[v] = new LinkedList<Integer>();
        }
        this.indegree = new int[nrOfVertices];
    }
}
```

Warum `indegree` als eigenes Feld? `outdegree(v)` ist schnell, weil `adj[v].size()` genügt. `indegree(v)` müsste sonst alle Listen durchsuchen. Mit `indegree[w]` wird der Wert beim Einfügen/Entfernen gepflegt.

9.9 Validierung von Knoten

Graphmethoden sollten prüfen, ob Knotennummern gültig sind:

```
private void validateVertex(int v) {
    if (v < 0 || v >= nrOfVertices) {
        throw new IllegalArgumentException("invalid vertex: " + v);
    }
}
```

Das verbindet Graphen mit [Kapitel 5](#): Fehlerhafte Eingaben sollten bei public Methoden nicht zu undefiniertem Verhalten führen.

9.10 addEdge und removeEdge

Für gerichtete Graphen fügt `addEdge(v, w)` eine Kante von `v` nach `w` ein.

```
public void addEdge(int v, int w) {
    validateVertex(v);
    validateVertex(w);
    adj[v].add(w);
    indegree[w]++;
    nrOfEdges++;
}
```

Entfernen:

```
public void removeEdge(int v, int w) {
    validateVertex(v);
    validateVertex(w);
    if (adj[v].remove((Integer) w)) {
        indegree[w]--;
        nrOfEdges--;
    }
}
```

Der Cast `(Integer) w` ist wichtig: Bei `List<Integer>` gibt es zwei `remove`-Varianten. Ohne Cast könnte `remove(int index)` gemeint sein, also Entfernen an Position `w`. Mit `(Integer) w` wird das Objekt mit Wert `w` entfernt.

9.11 degree, outdegree, indegree

```
public int outdegree(int v) {
    validateVertex(v);
    return adj[v].size();
}

public int indegree(int v) {
    validateVertex(v);
    return indegree[v];
}

public int degree(int v) {
    validateVertex(v);
    return outdegree(v) + indegree(v);
}
```

Bei ungerichteten Graphen ist Grad einfach die Anzahl angrenzender Kanten. Bei gerichteten Graphen ist der Grad die Summe aus Ein- und Ausgangsgrad.

9.12 reverse

`reverse` erzeugt einen neuen gerichteten Graphen mit umgedrehten Kanten.

```
public Digraph reverse() {
    Digraph r = new AdjListDigraph(nrOfVertices);
    for (int v = 0; v < nrOfVertices; v++) {
        for (int w : getAdj(v)) {
            r.addEdge(w, v);
        }
    }
    return r;
}
```

Wenn der ursprüngliche Graph eine Kante `(v, w)` enthält, enthält der `reverse`-Graph `(w, v)`.

Tiefe Idee: `reverse` zeigt, warum `getAdj(v)` eine saubere Schnittstelle ist. Die Methode muss nicht wissen, wie die Listen intern gespeichert sind.

9.13 Mini-Check Kapitel 9

1. Was ist bei einem gerichteten Graphen der Unterschied zwischen `InDeg` und `OutDeg`?

`InDeg` zählt eingehende Kanten, `OutDeg` ausgehende.

2. Wann ist eine Adjazenzmatrix sinnvoll?

Wenn sehr viele Kanten existieren oder sehr schnelle Kantenabfragen wichtig sind.

3. Warum verwendet `removeEdge` den Cast `(Integer) w`?

Damit `List.remove(Object)` statt `List.remove(int index)` verwendet wird.

4. Was macht `reverse`?

Es erstellt einen Graphen, in dem alle Kantenrichtungen umgedreht sind.

| Kapitel 10 - ausgelassen: Backtracking

Thema 10 wird nicht ausgearbeitet. Nach `PRO_Klausurinfo.md` kommt Backtracking gar nicht, auch nicht theoretisch. Es wird daher bewusst ausgelassen, damit die Lernzeit auf klausurrelevante Inhalte konzentriert bleibt.

Kapitel 11 - Objektorientierter Entwurf

[Zurück zur Kapitelübersicht](#)

11.1 Klausurfokus

Dieses Kapitel ist wichtig, weil es OOP-Wissen in Entwurfsaufgaben überführt. Nach `PRO_Klausurinfo.md` ist relevant:

- aus einem Text einen ersten Klassenentwurf machen,
- UML-Klassendiagramme lesen und zeichnen,
- Klassen, Datenkomponenten, Methoden und Beziehungen erkennen,
- Faktorisierung verstehen,
- Composite Pattern kennen,
- andere Entwurfsmuster nicht vertiefen.

Die Klausuraufgabe wird wahrscheinlich klein sein. Entscheidend ist nicht, einen perfekten Industrieentwurf zu liefern, sondern begründet Klassen und Beziehungen zu identifizieren.

11.2 Objektorientierter Entwurf: andere Denkweise

Beim aufgabenorientierten Entwurf startet man oft mit Algorithmen: Was ist zu tun? Beim objektorientierten Entwurf startet man mit Objekten: Welche Dinge der Problemwelt haben Zustand und Verhalten?

Typische Schritte:

1. Geeignete Klassen finden.
2. Operationen der Objekte festlegen.
3. Beziehungen zwischen Klassen bestimmen.
4. Gemeinsamkeiten faktorisieren.
5. Methoden algorithmisch entwerfen.

Tiefe Idee: Ein guter OO-Entwurf reduziert die semantische Lücke zwischen Problemwelt und Implementierung. Klassen sollen nicht nur Datencontainer sein, sondern sinnvolle Verantwortlichkeiten kapseln.

11.3 UML-Basics für die Klausur

Eine Klasse wird in UML meist als Rechteck mit drei Bereichen dargestellt:

```
+-----+
|  ClassName  |
+-----+
| - field: Type |
+-----+
| + method(p: T): R |
+-----+
```

Sichtbarkeit:

```
+ public
- private
# protected
~ package visibility
```

Vererbung:

```
Subclass ----|> Superclass
```

Interface-Realisierung:

```
Class ..|> Interface
```

Assoziation/Aggregation/Komposition werden in Klausuraufgaben meist einfacher behandelt als in vollständiger UML. Wichtig ist: Beziehung erklären können.

- Assoziation: Objekte kennen/verwenden einander.
- Aggregation: Ganzes-Teil-Beziehung, Teil kann auch unabhängig existieren.
- Komposition: stärkere Ganzes-Teil-Beziehung, Teil gehört lebenszyklisch zum Ganzen.

11.4 Klassen aus Text finden: Abbott-Heuristik

Die Folien beschreiben ein Vorgehen in Anlehnung an Abbott:

1. Verbale Spezifikation der Aufgabe aufstellen.
2. Klassen, Methoden und Datenkomponenten identifizieren:
 - Hauptwörter als Kandidaten für Klassen,
 - Zeitwörter als Kandidaten für Methoden,
 - Eigenschaftswörter als Kandidaten für Datenkomponenten.
3. Geeignete Basisklassen suchen und Gemeinsamkeiten faktorisieren.
4. Klassenspezifikationen aufstellen.
5. Methoden entwerfen.

Beispieltext:

Ein Roboter soll auf einer geraden Linie von Position p1 zur Position p2 zwei Karosserieteile durch Punktschweißung verbinden.

Mögliche Kandidaten:

Wort	Kandidat?	Begründung
Roboter	Klasse Robot	aktives Objekt mit Position und Operationen
Position	Klasse Position	Wertobjekt mit Koordinaten
Karosserieteil	Klasse BodyPart	fachliches Objekt
Punktschweißung	eher Methode/Werkzeug	Verfahren, eventuell Klasse SpotWeldingTool
verbinden	Methode	Operation

Achtung: Die Heuristik darf nicht blind angewandt werden. Nicht jedes Hauptwort ist eine Klasse, und nicht jedes Verb gehört als Methode in die zuerst gefundene Klasse.

11.5 Verantwortlichkeiten statt Datenhaufen

Ein häufiger Entwurfsfehler ist eine „Gottklasse“, die alle Daten kennt und alle Aktionen ausführt. Objektorientiert besser ist, Verantwortlichkeiten zu verteilen.

Schwächerer Entwurf:

```
class Robot {
    void spotWelding(BodyPart a, BodyPart b, Position p1, Position p2) { ... }
    void drill(BodyPart a, Position p) { ... }
    void paint(BodyPart a, Color c) { ... }
}
```

Problem: Jede neue Werkzeugart ändert Robot.

Besser:

```
abstract class RobotTool {
    abstract void apply(Position p1, Position p2);
}

class SpotWeldingTool extends RobotTool {
    void apply(Position p1, Position p2) { ... }
}
```



```

}

class Robot {
    private RobotTool tool;

    public void setTool(RobotTool tool) {
        this.tool = tool;
    }

    public void work(Position p1, Position p2) {
        tool.apply(p1, p2);
    }
}

```

Jetzt kann man neue Werkzeuge hinzufügen, ohne Robot jedes Mal umzubauen. Das ist Erweiterbarkeit durch Polymorphismus und Delegation.

11.6 Faktorisierung

Faktorisierung bedeutet: Gemeinsamkeiten verwandter Klassen werden in eine gemeinsame Basisklasse oder Schnittstelle gezogen.

Beispiel:

```

Circle, Rectangle, Line haben alle:
- Position
- draw()
- moveBy(dx, dy)

```

Möglicher Entwurf:

```

public abstract class Shape {
    private int x;
    private int y;

    public void moveBy(int dx, int dy) {
        x += dx;
        y += dy;
    }

    public abstract void draw();
}

```

Konkrete Klassen implementieren dann, was unterschiedlich ist:

```

public class Circle extends Shape {
    public void draw() { ... }
}

public class Rectangle extends Shape {
    public void draw() { ... }
}

```

Faktorisierung ist nicht Selbstzweck. Sie ist sinnvoll, wenn sie Redundanz reduziert, gemeinsame Schnittstellen schafft oder Erweiterbarkeit verbessert. Zu frühe oder falsche Faktorisierung kann den Entwurf komplizierter machen.

11.7 Beziehungen: ist-ein vs. hat-ein

Die wichtigste Entwurfsentscheidung ist oft: Vererbung oder Zusammensetzung?

Circle ist ein Shape.	-> Vererbung
Robot hat ein RobotTool.	-> Aggregation/Komposition
Car besteht aus Wheels.	-> Komposition/Aggregation

Prüffragen:

- Kann ein Objekt der Unterklasse überall verwendet werden, wo die Oberklasse erwartet wird? Dann ist Vererbung plausibel.

- Soll Verhalten zur Laufzeit austauschbar sein? Dann ist ein „hat-ein“-Entwurf oft besser.
- Enthält ein Objekt andere Objekte als Teile? Dann ist Aggregation/Komposition plausibel.

Klausurtyp: Wenn unsicher, Annahmen explizit notieren. Das ist laut Klausurinfo erlaubt bzw. sinnvoll.

11.8 Composite Pattern: Zweck

Das Composite Pattern ist das einzige Entwurfsmuster, das laut Klausurinfo ausdrücklich bekannt sein soll.

Zweck:

- Teil-Ganzes-Hierarchien darstellen,
- elementare und zusammengesetzte Objekte gleich behandeln,
- Client-Code vereinfachen.

Beispielhafte Problemstellung: Ein Grafiksystem soll einzelne Linien/Kreise, aber auch Gruppen von Formen zeichnen. Der Client soll nicht unterscheiden müssen, ob er eine einzelne Form oder eine Gruppe vor sich hat.

11.9 Composite Pattern: Struktur

Strukturmuster: *Composite*

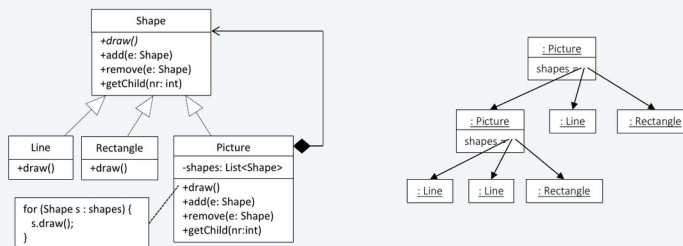


Zweck

- bilde Teil-Ganzes-Hierarchien von Objekten
- behandle elementare und komplexe Objekte gleich

Motivation

- Grafikeditor mit verschiedenen grafischen Objekten (Linie, Rechteck, ...), die zu Bildern zusammengefasst werden können



J. Pichler, V. 11.1, 2026

Seite 44

Composite Pattern mit Component, Leaf und Composite. Quelle: 11 OO Entwurf, Folie 44.

Typische Struktur:

```
public interface Component {
    void operation();
}

public class Leaf implements Component {
    public void operation() {
        // elementare Aktion
    }
}

public class Composite implements Component {
    private List<Component> children = new LinkedList<>();

    public void add(Component c) {
        children.add(c);
    }

    public void remove(Component c) {
        children.remove(c);
    }

    public void operation() {
        for (Component c : children) {

```

```

        c.operation();
    }
}

```

Client-Code:

```

public static void drawAll(Component c) {
    c.operation();
}

```

Der Client muss nicht wissen, ob `c` ein Blatt oder ein zusammengesetztes Objekt ist.

11.10 Composite-Beispiel: 3D-Drucker/Teilehierarchie

Für einen 3D-Drucker können Teile aus Unterteilen bestehen. Ein elementares Teil kann direkt gezeichnet werden. Ein zusammengesetztes Teil zeichnet alle enthaltenen Unterteile.

```

public abstract class Part {
    public abstract void drawTop();
}

public class Sphere extends Part {
    public void drawTop() {
        // Kreisprojektion zeichnen
    }
}

public class ComplexPart extends Part {
    private List<Part> parts = new LinkedList<>();

    public void add(Part p) {
        parts.add(p);
    }

    public void drawTop() {
        for (Part p : parts) {
            p.drawTop();
        }
    }
}

```

Der Vorteil ist dieselbe Nachricht an einfache und komplexe Objekte:

```

Part p = new ComplexPart();
p.drawTop();

```

Das ist dynamische Bindung aus [Kapitel 3](#) auf Entwurfsebene.

11.11 Vorgehen bei einer Entwurfsaufgabe in der Klausur

Ein zuverlässiges Vorgehen:

1. Aufgabenstellung zweimal lesen.
2. Fachliche Objekte markieren.
3. Kandidatenliste erstellen: Klassen, Methoden, Datenkomponenten.
4. Offensichtliche falsche Klassen streichen.
5. Beziehungen festlegen: Vererbung, Assoziation, Aggregation/Komposition.
6. Gemeinsame Eigenschaften faktorisieren.
7. UML-Skizze zeichnen.
8. Annahmen notieren, wenn die Angabe nicht eindeutig ist.

Mini-Beispiel:

Ein Zeichenprogramm verwaltet Kreise und Rechtecke. Beide können gezeichnet und verschoben werden. Außerdem können Gruppen von Formen gebildet werden, die ebenfalls gezeichnet und verschoben werden können.

Sehr guter Entwurf:

```
Shape {abstract}
+ draw()
+ moveBy(dx:int, dy:int)

Circle extends Shape
Rectangle extends Shape
Group extends Shape
- children: List<Shape>
+ add(s: Shape)
+ draw()
+ moveBy(dx:int, dy:int)
```

Warum gut? Group ist selbst ein Shape. Damit entsteht Composite: Einzelobjekte und Gruppen können gleich behandelt werden.

11.12 Mini-Check Kapitel 11

1. Sind alle Hauptwörter automatisch Klassen?

Nein. Sie sind nur Kandidaten.

2. Wann ist Vererbung passend?

Wenn eine echte ist-ein-Beziehung besteht und die Unterklasse überall als Oberklasse verwendbar ist.

3. Was ist Faktorisierung?

Das Herausziehen gemeinsamer Eigenschaften/Operationen in eine Basisklasse oder Schnittstelle.

4. Was ist der Kern des Composite Patterns?

Einheitliche Behandlung von Blättern und zusammengesetzten Objekten über eine gemeinsame Komponente.

Gesamt-Prüfungsstrategie

Für eine sehr gute Note sollten nicht nur Definitionen gelernt werden. Die Beispielklausur und die Klausurinfo zeigen klar: Die Aufgaben prüfen, ob man kurze Programme und Strukturen **auf Papier sicher durchdenken** kann.

A. Die wichtigsten aktiven Übungsformen

1. **Speicherbilder zeichnen:** Arrays, Objekte, Strings, Aliasing.
2. **Kurze Methoden schreiben:** `join`, `count`, `contains`, `max`, `reverse`, `parse`.
3. **Einfache Klassen implementieren:** Konstruktoren, Objektmethoden, `equals`, `hashCode`, Invarianten.
4. **Dynamische Bindung tracen:** statischer Typ, dynamischer Typ, überschrieben vs. überladen.
5. **ATG entwickeln:** Grammatik zuerst, dann Attribute und semantische Aktionen.
6. **Exceptions tracen:** welcher `catch`, läuft `finally`, geht es danach weiter?
7. **Hashing erklären:** Kollision, Lastfaktor, Kollisionsbehandlung.
8. **Heap zeichnen:** Feld zu Baum, Baum zu Feld, `swim/sink`.
9. **Graphmethoden schreiben:** `addEdge`, `removeEdge`, `degree`, `reverse`.
10. **OO-Entwurf skizzieren:** Klassen aus Text, UML, Faktorisierung, Composite.

B. Priorisierung nach Klausurwahrscheinlichkeit

Sehr hoch:

- Felder/Referenzen und kleine Klassenmethoden,
- einfache Klassen mit Konstruktoren und Methoden,
- Vererbung, dynamische Bindung, `equals/hashCode`,
- ATG/LL(1),
- Ausnahmebehandlung,
- Heap-Grundoperationen,
- OO-Entwurf mit UML/Composite.

Hoch:

- Hashing-Grundideen,
- Graphen mit Adjazenzlisten,
- Pakete/Sichtbarkeit,
- Bibliotheksmethoden wie `Integer.parseInt`.

Reduziert:

- Generics nur minimal,
- Heapsort nicht,
- Backtracking nicht,
- andere Design Patterns nicht vertiefen.

C. Strategie in der 90-Minuten-Klausur

- Aufgaben erst vollständig lesen.
- Bei Codingaufgaben zuerst Signatur und Randfälle klären.
- Bei Speicherbildern sauber zwischen Variable, Referenz und Objekt/Array unterscheiden.
- Bei `equals` immer `Object` als Parameter verwenden.
- Bei ATG zuerst die reine Syntax lösen, dann Attribute ergänzen.
- Bei Heap Aufgaben Index 0 sofort als frei markieren.
- Bei Graphen klären, ob gerichteter oder ungerichteter Graph gemeint ist.

- Bei unklarer Angabe eine plausible Annahme notieren und konsistent weiterarbeiten.

| Finale Checkliste gegen typische Punkteverluste

Java-Grundlagen und Felder

- `String` mit `.equals` vergleichen, nicht versehentlich mit `==`.
- Vor `s.equals(...)` bedenken, ob `s` `null` sein kann.
- Lokale Variablen explizit initialisieren.
- Defaultwerte von Arrays und Datenkomponenten kennen.
- Arrayvariable enthält Referenz, nicht das Array selbst.
- Java übergibt immer call-by-value; bei Referenztypen wird der Referenzwert kopiert.

Klassen/OOP

- Konstruktor hat keinen Rückgabotyp.
- `this(...)` oder `super(...)` muss erste Anweisung im Konstruktor sein.
- `private` ist klassenbezogen, nicht objektbezogen.
- Überschreiben: gleiche Signatur in Unterklasse.
- Überladen: gleicher Name, andere Parameterliste.
- Dynamische Bindung richtet sich nach dynamischem Typ.
- Statischer Typ bestimmt, was der Compiler als sichtbar erlaubt.
- `equals(Object o)` korrekt überschreiben, nicht `equals(MyClass x)`.
- Bei `equals`: Typcheck mit `instanceof`, dann Cast.
- `hashCode` passend zu `equals` überschreiben.

ATG und Grammatiken

- Jede Grammatikregel mit Punkt abschließen.
- Terminale und Terminalklassen unterscheiden.
- `ident` und `number` sprachlich oder per Regex beschreiben können.
- LL(1)-Konflikte durch gemeinsamen Präfix erkennen.
- Beispiel `B = C | C "." C` umformen zu `B = C [ "." C ]`.
- Syntax vor Semantik lösen.
- Attribute an den Stellen einsetzen, an denen Werte wirklich bekannt sind.

Exceptions

- `finally` wird ausgeführt, auch wenn eine Exception auftritt.
- Nach unbehandelter Exception läuft die Methode nicht normal weiter.
- Spezifische catch-Blöcke vor allgemeineren platzieren.
- Checked Exceptions behandeln oder mit `throws` deklarieren.
- `RuntimeExceptions` müssen nicht deklariert werden.

Hashing

- Kollisionen sind unvermeidbar.
- Verkettung und offene Adressierung unterscheiden können.
- Lastfaktor $\alpha = n / m$ erklären können.
- 0(1) gilt nur erwartbar bei guter Verteilung und kontrolliertem Füllstand.
- Gleiche Objekte nach `equals` müssen gleichen Hashcode haben.

Heap

- Position 0 im Feld freilassen.
- Wurzel steht bei Index 1.
- Links $2i$, rechts $2i+1$, Parent $i/2$.
- Heap ist nicht vollständig sortiert.
- `insert` nutzt `swim`.
- `removeMax` nutzt Tausch mit letztem Element und `sink`.
- Bei `sink` mit größerem Kind tauschen.

Graphen

- Gerichtete Kante (v, w) ist nicht dasselbe wie (w, v) .
- `outdegree` zählt ausgehende Kanten.
- `indegree` zählt eingehende Kanten.
- Adjazenzmatrix: schneller Zugriff, hoher Speicher.
- Adjazenzliste: speichereffizient, Kantenabfrage langsamer.
- Bei `remove((Integer) w)` den Unterschied zu `remove(int index)` kennen.

OO-Entwurf

- Hauptwörter sind nur Klassenkandidaten.
- Verben sind nur Methodenkandidaten.
- Klassen sollten Verantwortlichkeiten kapseln, nicht nur Daten sammeln.
- `ist-ein` spricht für Vererbung.
- `hat-ein/besteht-aus` spricht für Aggregation/Komposition.
- Faktorisierung nur bei echten Gemeinsamkeiten.
- Composite: Leaf und Composite über gemeinsame Component gleich behandeln.
- Bei unklaren Angaben Annahmen explizit notieren.